Cours : Reverse Engineering et Sécurité Globale

Chapitre 1 : Reverse Engineering
Le reverse engineering (ou rétro-ingénierie) est le processus d'analyse d'un système ou d'un logiciel compilé pour en comprendre le fonctionnement interne, sans en posséder le code source. Cette discipline est essentielle pour décortiquer des malwares, rechercher des vulnérabilités ou comprendre la logique d'un programme.

L'Analyse Statique
L'analyse statique consiste à étudier la structure du programme sans l'exécuter. Le fichier binaire est traduit en code assembleur ou en pseudo-code pour en lire la logique mathématique. Cette méthode utilise des désassembleurs et décompilateurs (comme Ghidra ou IDA Pro) pour repérer les fonctions clés, lire les chaînes de caractères et inspecter les sections du fichier avant tout risque d'exécution.

L'Analyse Dynamique
L'analyse dynamique permet d'observer le comportement réel du logiciel pendant son exécution. Elle s'effectue à l'aide d'un débogueur (comme x64dbg ou OllyDbg) qui permet d'exécuter le programme pas-à-pas, de lire la mémoire en temps réel et de placer des points d'arrêt. Pour des raisons de sécurité, notamment lors de l'étude de malwares, cette analyse doit être réalisée dans un environnement strictement isolé (machine virtuelle).

Chapitre 2 : Forensic Numérique
Le forensic numérique (ou criminalistique numérique) se rapporte à toute analyse scientifique spécifique aux systèmes d'information, aux appareils électroniques et aux données numériques. Tout comme la criminalistique traditionnelle, l'objectif est d'identifier, de préserver, d'analyser, de documenter et de présenter le résultat de ces opérations devant la justice.

Les 5 étapes de l'investigation
1. **Identification** : Repérer les sources potentielles de preuves numériques.
2. **Préservation** : Sécuriser les données (mise sous scellés, copie bit-à-bit) pour garantir qu'elles ne soient pas altérées.
3. **Analyse** : Examiner les copies de données pour en extraire des éléments probants.
4. **Documentation** : Noter scrupuleusement chaque action menée.
5. **Présentation (Restitution)** : Exposer les résultats de l'enquête de manière intelligible pour les tribunaux.

Les principes fondamentaux
- **Le Hash (Empreinte numérique)** : C'est l'ADN du programme ou du fichier. S'il est modifié, même d'un seul bit, son ADN (le hash) change complètement. Il est là pour sécuriser la preuve, vérifier l'intégrité des données et détecter si les programmes ont été bricolés.
- **Règle d'or** : Il faut toujours garder une trace écrite de toutes les opérations pour maintenir la validité juridique de la chaîne de possession de la preuve.

Chapitre 3 : Cadre Légal et Acteurs de la Justice

Toute investigation numérique doit respecter un cadre légal strict pour que les preuves soient recevables au tribunal.

Hiérarchie des Normes (Pyramide de Kelsen)
L'ordre juridique français est structuré de manière pyramidale. De la norme suprême à la norme la plus faible :
1. **Bloc Constitutionnel** : L'ensemble des principes et dispositions que doivent respecter toutes les lois.
2. **Bloc Conventionnel** : Traités, conventions internationales et Droit Européen.
3. **Bloc Législatif** : Lois organiques, lois ordinaires, ordonnances.
4. **Principes généraux du droit** : Règles non écrites de portée générale.
5. **Bloc Réglementaire** : Décrets, arrêtés, circulaires.

Les différents types d'enquête
- **Enquête de Flagrance** (article 53 et suivants du Code de Procédure Pénale - CPP). C'est le régime de "l'urgence". Il s'applique lorsqu'un crime ou un délit est "en train de se commettre" ou "vient de se commettre".
- **Enquête Préliminaire** (article 75 du CPP). C'est le régime "par défaut" lorsqu'il n'y a pas d'urgence ou de flagrance. Elle est dirigée par le Procureur de la République suite à une plainte, un signalement, ou à l'initiative de la police.
- **Commission Rogatoire** (article 151 et suivants du CPP). Ce cadre s'applique lors d'une "information judiciaire", c'est-à-dire dans les affaires criminelles ou complexes confiées à un Juge d'instruction.
- **Recherche des causes de la mort** (article 74 du CPP). Il s'agit d'un cadre d'enquête spécial (sui generis) utilisé lorsqu'un cadavre est découvert, qu'une personne est grièvement blessée, et que les causes sont inconnues ou suspectes (suicide, accident suspect, meurtre potentiel).
- **Disparition Inquiétante** (article 74-1 du CPP). Un autre cadre spécial utilisé lorsqu'une disparition récente présente un caractère inquiétant (mineur, majeur protégé, personne malade, ou circonstances suspectes).

Les acteurs de la procédure
- **Les Magistrats** : Les procureurs (qui dirigent les enquêtes), les juges d'instruction, ainsi que les magistrats du siège et spécialisés.
- **Les Forces de l'ordre** : Les Officiers de Police Judiciaire (OPJ) qui disposent de pouvoirs étendus d'investigation, les Agents de Police Judiciaire (APJ) qui secondent les OPJ, les Agents de Police Judiciaire Adjoints (APJA) et les Agents de la Force Publique. [cabinetaci](https://www.cabinetaci.com/les-agents-de-police-judiciaire/)
- **Agences spécialisées** : La JUNALCO (Juridiction Nationale de Lutte contre la Criminalité Organisée) et l'IRCGN (Institut de Recherche Criminelle de la Gendarmerie Nationale).
- **Les Intervenants Qualifiés** : Les enquêteurs spécialisés, les experts, les analystes et les techniciens.
- **Modes de désignation** : Les experts sont nommés par Ordonnance de Commission d'expert (par le juge d'instruction) ou par réquisition judiciaire (Article 60 du CPP).

Chapitre 4 : Architecture Numérique et Conversions

La compréhension des données à bas niveau nécessite de maîtriser les systèmes de numération.

Les bases de conversions
- **Décimal vers Binaire** : On utilise la méthode des divisions successives par 2, en conservant les restes de bas en haut pour former le nombre binaire.
- **Binaire vers Hexadécimal** : On utilise la méthode du regroupement. On sépare les bits par paquets de quatre (quartets) en partant de la droite, puis on convertit chaque groupe en son équivalent hexadécimal (ex: 1010 = A).
[fr.wikihow](https://fr.wikihow.com/convertir-du-binaire-en-hexad%C3%A9cimal)
- **Hexadécimal vers Binaire** : Chaque chiffre ou lettre de la valeur hexadécimale est retranscrit directement dans son équivalent binaire de 4 bits.

Chapitre 5 : Cryptographie
La cryptographie est l'art de chiffrer des messages pour en protéger la confidentialité.

Le Chiffre de César
C'est une méthode de chiffrement où les lettres du texte sont remplacées par d'autres lettres selon un décalage fixe dans l'alphabet. Ce système est vulnérable à l'analyse de fréquence des lettres dans la langue ciblée (ex: le "E" en français), ce qui permet de retrouver les mots et la clé de décalage en convertissant les occurrences.

Le Chiffre de Vigenère
C'est un chiffrement par substitution qui utilise un mot-clé pour définir les décalages. Chaque lettre de la clé correspond à un décalage différent dans l'alphabet. Cette méthode permet de lisser l'apparition des lettres dans le texte chiffré, rendant l'analyse fréquentielle (qui casse le César) inefficace.

Chapitre 6 : Bases d'Électronique
L'analyse matérielle et le forensic sur des composants nécessitent des bases en électronique.

Grandeurs physiques et Puissance
- **Tension (U)** : mesurée en Volts (V).
- **Intensité (I)** : mesurée en Ampères (A).
- **Résistance (R)** : mesurée en Ohms (Ω).
- **Puissance (P)** : Le calcul de la puissance se fait via la formule $P = U \times I$.

Les lois de l'électricité
- **Loi des nœuds** : L'électricité se divise lorsqu'il y a scission du circuit électrique. La somme des intensités des courants arrivant à un nœud est égale à la somme des intensités des courants qui en repartent ($I = I_1 + I_2 + I_3$).
[maxicours](https://www.maxicours.com/se/cours/loi-des-noeuds-et-loi-des-mailles/)
- **Équation des résistances** :
  - **En série** : Les résistances s'additionnent. Exemple pour deux résistances de $1 k\Omega$: ($R_{eq} = 1 k\Omega + 1 k\Omega = 2 k\Omega$).

- **En parallèle** : La résistance équivalente diminue. Équation : $\frac{1}{R_{eq}} = \frac{1}{R_1} + \frac{1}{R_2}$. Pour deux résistances de $1 k\Omega$, $R_{eq} = 500 \Omega$.
- **Pont diviseur de tension** : Ce montage de deux résistances en série permet de répartir une tension. La tension aux bornes des deux résistances sera celle de la source, et elle se répartit proportionnellement à leurs valeurs selon la formule : $U_{R2} = U \times \frac{R_2}{R_1 + R_2}$.

Outil de simulation : *Falstad* est un site web recommandé pour créer et visualiser des montages de circuits électroniques.

l'IRCGN (Institut de Recherche Criminelle de la Gendarmerie Nationale), qui est le grand pôle de la police scientifique française. L'IRCGN regroupe l'expertise "traditionnelle" (comme la balistique) et l'expertise "numérique/informatique".

Voici comment ces différents départements sont organisés et ce qu'ils font :

1. La Division Criminalistique Ingénierie et Numérique (DCIN)
C'est ici que se trouve le cœur de l'expertise informatique et électronique "matérielle" de la gendarmerie scientifique au sein de l'IRCGN. Elle intervient pour extraire des données sur des supports physiques (téléphones, disques durs, calculateurs de voitures). Elle comprend plusieurs départements :

Le Département Informatique et Électronique (INL) : Ses experts démontent, réparent et extraient les données des disques durs endommagés, des téléphones détruits ou chiffrés. Ils font beaucoup de forensic matériel (lecture directe de puces mémoire, chip-off).

Le Département Signal, Image, Parole (DSIP) / Département Document Numérique Multimédia (DDNM) : C'est le fameux "département d'analyse" audiovisuelle. Ils analysent la voix (reconnaissance vocale), nettoient des bandes audios ou vidéos de vidéosurveillance pour identifier un suspect, et traquent les trucages et deepfakes.

Le Département Véhicules : Ils analysent l'électronique embarquée des voitures (ordinateurs de bord, calculateurs) pour extraire des données GPS ou comprendre les causes d'un accident.

2. Le COMCYBER-GEND et le C3N (La vraie "Gendarmerie Informatique")
Au-delà de l'IRCGN, la lutte contre la cybercriminalité pure (piratage, ransomware, pédopornographie en ligne, dark web) est gérée par le Commandement de la Gendarmerie dans le Cyberespace (COMCYBER-GEND), dont le fer de lance est le C3N (Centre de Lutte Contre les Criminalités Numériques). Le C3N possède ses propres départements d'analyse numérique :

Le Département d'Analyse (Coordination et Renseignement) : Il analyse les phénomènes criminels émergents, étudie les modes opératoires des hackers et identifie les menaces globales.

Le Centre National d'Analyse des Images de Pédopornographie (CNAIP) : Il analyse et classifie des millions d'images pour identifier et secourir des victimes.

Le Département Technique : Il développe des outils d'investigation, d'analyse de malwares et de reverse engineering pour les enquêteurs cyber.

3. Le Département Balistique (Division Physique-Chimie)
Ce département n'a rien à voir avec l'informatique, il fait partie de la division Physique-Chimie de l'IRCGN.

Son rôle : Les experts balisticiens étudient les armes à feu, les munitions et les trajectoires de tir.

L'analyse : Ils comparent les résidus de tirs, examinent les rayures laissées sur une balle par le canon d'une arme (chaque arme laisse une signature unique, un peu comme une empreinte digitale) pour relier une balle retrouvée sur une scène de crime à l'arme du tireur. Ils peuvent également modéliser des scènes de crime en 3D pour reconstituer la trajectoire exacte d'un projectile.

En résumé pour ton cours : si tu parles de forensic numérique, concentre-toi sur la DCIN de l'IRCGN et le C3N. La balistique relève de la criminalistique physique.

Initialisation à la cryptographie

1. Qu'est-ce que la cryptographie ?
La cryptographie est l'art et la science de protéger des informations en les transformant (chiffrement) pour les rendre illisibles à toute personne ne possédant pas la clé permettant de les ramener à leur état initial (déchiffrement). En cybersécurité, elle est le pilier central de la protection des données et des communications.

2. Les objectifs fondamentaux de la cryptographie
La cryptographie moderne ne sert pas uniquement à cacher un message. Elle répond à plusieurs objectifs cruciaux de sécurité, souvent résumés par les critères suivants :

La Confidentialité : C'est le but premier. Il s'agit de s'assurer que seuls le destinataire et le lecteur légitimement autorisé possèdent la clé pour lire le message. Tout intercepteur ne verra qu'une suite de caractères incompréhensible.

L'Authentification (et l'Intégrité) : Il est indispensable de pouvoir vérifier si le message vient bel et bien de l'émetteur prétendu (authentification) et de s'assurer qu'il n'a pas été modifié ou altéré en cours de route par un attaquant (intégrité).

La Non-répudiation : L'émetteur d'un message chiffré ou signé numériquement ne peut pas nier l'avoir envoyé plus tard.

3. Un peu d'histoire : Le chiffrement de César
La cryptographie existe depuis l'Antiquité. L'un des algorithmes historiques les plus connus est le chiffre de César.

Le principe : Utilisé par Jules César pour ses correspondances militaires, il s'agit d'un chiffrement par substitution monoalphabétique. Chaque lettre du message clair est remplacée par une autre lettre située plus loin dans l'alphabet, selon un décalage fixe (la clé). Par exemple, avec un décalage de 3, le "A" devient "D", le "B" devient "E", etc.

La faille : Ce système est aujourd'hui obsolète et extrêmement vulnérable. Comme le décalage est le même pour tout le message, il suffit de faire une "analyse fréquentielle" (repérer la lettre la plus fréquente du texte chiffré et déduire qu'il s'agit du "E", la lettre la plus courante en français) pour retrouver la clé et lire le message.

4. Le concept du Masque Jetable (OTP - One-Time Pad)
Le Masque Jetable (ou One-Time Pad en anglais) est un algorithme inventé en 1917 par Gilbert Vernam. C'est le seul système de chiffrement prouvé mathématiquement incassable en théorie, car il résiste à toutes les analyses cryptanalytiques (et même à la puissance des ordinateurs quantiques).

Il fonctionne en combinant le message clair avec une clé (le masque). Souvent, en informatique moderne, cette combinaison se fait via une opération mathématique appelée "XOR" (OU exclusif) bit par bit.

Pour que l'OTP soit véritablement incassable, il faut respecter 3 règles strictes :

La longueur de la clé : La clé doit être exactement de la même longueur ou plus longue que le message à chiffrer.

Le côté aléatoire : Les caractères (ou les bits) composant la clé doivent être générés de manière purement et strictement aléatoire.

L'usage unique (Jetable) : La clé ne doit être utilisée qu'une seule et unique fois pour chiffrer un seul message. Si la clé est réutilisée, le chiffrement devient vulnérable et peut être cassé.

La limite de l'OTP : Bien que parfait en théorie, il est très complexe à utiliser dans la vraie vie. Le fait de devoir générer et transmettre au destinataire de manière totalement sécurisée une clé aléatoire aussi longue que le fichier à envoyer pose un immense problème de logistique (le "problème de l'échange des clés").

Chapitre 2 : L'évolution de la cryptographie (Historique)

1. Le chiffre de Vigenère et sa clé

Pour contrer la grande faiblesse du chiffre de César (qui se casse facilement avec l'analyse fréquentielle des lettres), le Français Blaise de Vigenère a popularisé au XVIe siècle un chiffrement beaucoup plus robuste : le chiffrement polyalphabétique.

Le principe : On utilise un mot-clé (la clé de Vigenère) que l'on répète en boucle sous le texte à chiffrer. Chaque lettre de cette clé va déterminer un décalage différent dans l'alphabet (souvent visualisé grâce au "carré de Vigenère").

L'avantage : Une même lettre dans le message clair (ex: le "E") sera chiffrée par des lettres différentes selon sa position, ce qui rend l'analyse fréquentielle classique totalement inefficace. Il faudra attendre le XIXe siècle (avec Charles Babbage et Friedrich Kasiski) pour que des mathématiciens trouvent une méthode pour le casser.

2. La machine Enigma et Alan Turing

Pendant la Seconde Guerre mondiale, l'armée allemande utilisait la machine Enigma, un dispositif électromécanique à rotors extrêmement complexe pour chiffrer ses communications.

Le fonctionnement : Chaque fois qu'une lettre était tapée, les rotors tournaient, modifiant continuellement le circuit électrique et donc l'alphabet de substitution. Une même lettre tapée deux fois de suite donnait deux résultats différents.

Le rôle d'Alan Turing : Le mathématicien britannique Alan Turing (à Bletchley Park) a repris et amélioré les travaux de mathématiciens polonais pour concevoir les "Bombes", d'immenses machines capables de tester des milliers de combinaisons de rotors par seconde. En cassant le code d'Enigma, Turing a permis aux Alliés d'anticiper les mouvements allemands, raccourcissant la guerre de plusieurs années.

Turing complet : Bien que ce concept dépasse la cryptographie pour toucher à l'informatique théorique, Alan Turing est aussi le père de la machine de Turing. Un système (comme un langage de programmation) est dit "Turing complet" s'il possède une puissance de calcul suffisante pour simuler n'importe quelle machine de Turing (c'est-à-dire qu'il peut résoudre n'importe quel problème calculable, sous réserve de temps et de mémoire).

Chapitre 3 : La Cryptographie Moderne et les Algorithmes Symétriques

La cryptographie symétrique est un système où la même clé sert à la fois pour chiffrer et pour déchiffrer. L'algorithme le plus célèbre et utilisé aujourd'hui est l'AES (Advanced Encryption Standard).

L'algorithme AES (Advanced Encryption Standard)

L'AES est un algorithme de chiffrement symétrique par blocs (il découpe le message en blocs de 128 bits et les chiffre un par un). Pour brouiller les données, il applique plusieurs "tours" de chiffrement (rounds). Selon la taille de la clé (128, 192 ou 256 bits), il effectuera

10, 12 ou 14 tours (par exemple, tes notes mentionnaient 13 tours, ce qui correspond aux 13 tours complets de l'AES-256 avant le dernier tour simplifié).

Chaque tour complet applique 4 opérations mathématiques strictes dans l'ordre suivant :

SubBytes (Substitution) : Chaque octet du bloc est remplacé par un autre octet en utilisant une table de correspondance complexe (la S-Box).

ShiftRows (Décalage des lignes) : Les lignes de la matrice de données sont décalées vers la gauche (cela éparpille les données).

MixColumns (Mélange des colonnes) : Une opération mathématique mélange les colonnes de la matrice pour diffuser l'information massivement.

AddRoundKey (Ajout de la clé de tour) : On fait un "XOR" entre la matrice de données et une clé spécifique générée pour ce tour (dérivée de la clé maître).

Faire les étapes en inversé (InverseShiftRows, InverseSubBytes...) ne permet pas de deviner la clé. Faire les étapes à l'envers, c'est tout simplement le processus normal de déchiffrement effectué par le destinataire qui possède déjà la clé ! Un attaquant sans la clé ne peut pas remonter à l'envers.

Le VRAI problème des algorithmes symétriques, c'est la distribution de la clé. Puisque l'émetteur et le récepteur doivent utiliser la même clé secrète, comment faire pour s'échanger cette clé par internet (qui est non sécurisé) sans se la faire voler au passage ? Ce problème logistique majeur a conduit à l'invention de la cryptographie asymétrique (clé publique / clé privée, comme RSA ou ECC).

Chapitre 4 : La Cryptographie Asymétrique
Pour résoudre le grand problème de l'échange de clé sécurisé de la cryptographie symétrique (comment transmettre la clé secrète sans se la faire intercepter), des mathématiciens ont inventé la cryptographie asymétrique (ou chiffrement à clé publique) dans les années 1970.

Le principe des paires de clés
Contrairement au système symétrique (une seule clé), le système asymétrique repose sur une paire de clés mathématiquement liées :

La Clé Publique : Elle est distribuée librement à tout le monde. Elle sert uniquement à chiffrer un message.

La Clé Privée : Elle est gardée secrète par son propriétaire et ne doit jamais être partagée. Elle est la seule capable de déchiffrer le message qui a été chiffré par la clé publique correspondante.

Exemple pratique : Si Alice veut envoyer un message secret à Bob, Alice utilise la clé publique de Bob pour chiffrer le message. Une fois chiffré, même Alice ne peut plus le déchiffrer. Seul Bob, avec sa clé privée, pourra le lire.

Les algorithmes les plus connus sont RSA (basé sur la factorisation de grands nombres premiers) et ECC (basé sur les courbes elliptiques).

L'authentification et l'échange de clé aléatoire (Cryptographie Hybride)
Le chiffrement asymétrique est très sécurisé mais il demande énormément de puissance de calcul et s'avère très lent pour de gros fichiers. Pour y remédier, les systèmes modernes (comme HTTPS sur internet) utilisent la cryptographie hybride :

On génère d'abord une clé symétrique aléatoire et éphémère (appelée "clé de session").

On utilise la cryptographie asymétrique (très sûre) juste un court instant pour chiffrer et s'échanger cette petite clé de session aléatoire.

Une fois que l'émetteur et le récepteur partagent cette même clé de session, ils l'utilisent avec un algorithme symétrique rapide (comme l'AES) pour chiffrer tout le reste de leur communication.

Chapitre 5 : Le Hachage (Hashing)
Le hachage n'est pas du chiffrement ! Le chiffrement est réversible (on peut déchiffrer avec une clé), mais le hachage est une fonction mathématique à sens unique.

Le principe
Une fonction de hachage (comme SHA-256 ou MD5, bien que ce dernier soit obsolète) prend en entrée un fichier ou un texte de n'importe quelle taille (de 1 ko à 50 Go) et le transforme en une chaîne de caractères de taille fixe, appelée empreinte numérique (ou hash, ou digest).

Il est mathématiquement impossible de faire l'opération inverse pour retrouver le fichier d'origine à partir du hash.

À quoi ça sert ?
Le hachage est utilisé principalement pour deux choses :

L'intégrité des données : Si tu télécharges un fichier et qu'un seul bit a été modifié par un pirate (ou corrompu pendant le téléchargement), le hash du fichier sera radicalement différent. Comparer le hash permet de s'assurer de l'authenticité d'un fichier en forensic numérique ou lors d'un téléchargement.

Le stockage des mots de passe : Les sites web ne stockent jamais (ou ne devraient jamais stocker) ton mot de passe en clair dans leur base de données. Ils stockent uniquement son hash. Quand tu te connectes, le site hache ce que tu as tapé et compare le résultat avec le hash stocké en base de données. Si un pirate vole la base de données, il ne verra que des hashs illisibles et irréversibles.

Chapitre 6 : L'Informatique Quantique et ses menaces
L'informatique quantique repose sur les principes de la mécanique quantique (comme la superposition et l'intrication). Contrairement à un ordinateur classique qui utilise des bits (0 ou 1), un ordinateur quantique utilise des qubits, capables d'être à la fois 0 et 1. Cette puissance de calcul parallèle pose un risque mortel pour la cryptographie actuelle.

Il existe deux algorithmes quantiques théoriques qui menacent directement nos systèmes actuels :

L'algorithme de Shor (La menace asymétrique) : Cet algorithme permet de factoriser des nombres premiers gigantesques de manière exponentiellement plus rapide qu'un ordinateur classique. Le jour où un ordinateur quantique sera assez puissant, il pourra casser instantanément les chiffrements RSA et ECC (Courbes Elliptiques). Cela signifie que toute la cryptographie à clé publique actuelle est vouée à disparaître.

L'algorithme de Grover (La menace symétrique) : Cet algorithme permet de faire de la recherche par force brute ultra-rapide (accélération quadratique). Il divise techniquement par deux l'efficacité des clés symétriques et des hashs. Un chiffrement AES-128 ne vaudra plus qu'un AES-64 (et sera donc cassable). Pour se protéger, il suffit d'augmenter la taille des clés : il faudra passer à l'AES-256 et au SHA-384 ou SHA-512.

Chapitre 7 : La Cryptographie Post-Quantique (PQC)
La cryptographie post-quantique est la réponse logicielle à la menace de l'algorithme de Shor. Il s'agit de nouveaux algorithmes mathématiques complexes conçus pour tourner sur nos ordinateurs classiques actuels (téléphones, PC), mais qui sont prouvés comme étant résistants aux attaques d'un ordinateur quantique.

Le NIST (National Institute of Standards and Technology américain) a récemment finalisé la standardisation des premiers algorithmes PQC pour remplacer RSA et ECC dans le monde entier :

ML-KEM (anciennement CRYSTALS-Kyber) : C'est le nouveau standard (FIPS 203) pour l'échange de clés (encapsulation). Il va remplacer des protocoles comme l'ECDH utilisés dans nos connexions TLS (HTTPS), SSH et VPN. Il est basé sur la mathématique des réseaux euclidiens.

ML-DSA (anciennement CRYSTALS-Dilithium) : C'est le nouveau standard (FIPS 204) pour les signatures numériques (qui remplaceront les signatures RSA ou ECDSA utilisées dans les certificats électroniques).

HQC : Récemment sélectionné (mars 2026) comme algorithme de "secours" au ML-KEM. Il repose sur des mathématiques différentes (basées sur les codes correcteurs d'erreurs) au cas où une faille serait découverte un jour dans le ML-KEM.

Chapitre 8 : La Communication Quantique (QKD)
Là où la PQC est une solution logicielle/mathématique, la communication quantique est une solution matérielle et physique. Elle repose principalement sur le QKD (Quantum Key Distribution - Distribution quantique de clés).

Le principe
Le QKD ne sert pas à chiffrer la donnée en elle-même, mais à échanger une clé symétrique de manière absolument inviolable.

L'expéditeur encode des informations (des qubits) dans les propriétés physiques de particules de lumière (comme la polarisation des photons) et les envoie dans un câble à fibre optique.

La sécurité absolue garantie par la physique
Le gros avantage du QKD repose sur une loi fondamentale de la mécanique quantique (le principe d'incertitude / l'effet d'observation) : il est impossible d'observer ou de copier un état quantique sans l'altérer.

Si un pirate (une écoute clandestine) tente d'intercepter les photons pour lire la clé, la simple action de "regarder" va modifier la lumière.

À la réception, l'expéditeur et le destinataire comparent un sous-ensemble de leur transmission. S'ils constatent un taux d'erreur élevé, ils savent instantanément que la ligne est sur écoute. Ils jettent alors la clé et en génèrent une nouvelle.

Une fois la clé échangée en toute sécurité via le canal quantique (fibre optique ou satellite), les données réelles sont chiffrées de manière ultra-sécurisée avec un algorithme classique très robuste (comme l'AES-256).
Le chiffrement symétrique et le chiffrement asymétrique se distinguent principalement par le nombre de clés utilisées pour chiffrer et déchiffrer les données, ce qui impacte directement leur vitesse et leur niveau de sécurité. Alors que la cryptographie symétrique est rapide mais pose des problèmes de transmission de clé, la cryptographie asymétrique résout ce problème au prix d'une plus grande lourdeur de calcul.primx+1

Différences entre les chiffrements

Caractéristique	Chiffrement Symétrique	Chiffrement Asymétrique

Caractéristique	Chiffrement Symétrique	Chiffrement Asymétrique
Nombre de clés	1 seule clé partagée (clé secrète) [primx].	2 clés liées (clé publique et clé privée) [primx].
Fonctionnement	La même clé sert à la fois pour le chiffrement et le déchiffrement [hexn].	La clé publique chiffre le message, seule la clé privée peut le déchiffrer [primx].
Rapidité et ressources	Très rapide, nécessite peu de puissance de calcul informatique [primx].	Très lent, nécessite beaucoup de ressources de calcul en raison des mathématiques complexes primx+1.
Taille des clés	Courte (généralement 128 ou 256 bits) [primx].	Très longue (souvent 2048 ou 4096 bits pour compenser les vulnérabilités mathématiques) primx+1.
Volume de données	Idéal pour chiffrer de grandes quantités de données et des fichiers lourds primx+1.	Réservé aux très petites quantités de données (comme chiffrer un mot de passe ou une autre clé) [sealpath].
Authentification et Signature	Ne permet pas nativement de prouver l'identité de l'expéditeur de manière infaillible [primx].	Permet l'authentification et la signature électronique (grâce à la clé privée unique à l'expéditeur) hexn+1.
Vulnérabilité principale	La distribution de la clé : si elle est interceptée lors de l'échange, tout le système est compromis [hexn].	Moins vulnérable lors des échanges, mais très exposé si la clé privée est volée sur la machine de l'utilisateur [primx].
Algorithmes connus	AES, DES, ChaCha20.	RSA, ECC (Courbes elliptiques).

1. PKE (Public Key Encryption - Chiffrement à Clé Publique)
Le PKE est la forme la plus classique et directe de cryptographie asymétrique. Il sert à chiffrer directement un message de sorte que seul le destinataire légitime puisse le lire.

Comment ça marche : Alice possède une paire de clés (une clé publique accessible à tous, et une clé privée gardée secrète). Si Bob veut envoyer un message à Alice, il utilise la clé publique d'Alice pour chiffrer son message. Une fois chiffré, seul le détenteur de la clé privée (Alice) peut déchiffrer ce message.

Exemple concret : L'algorithme RSA. Tu veux envoyer un email sécurisé à ta banque contenant ton numéro de carte de crédit. Tu utilises la clé publique RSA de la banque pour chiffrer l'email. Même si un pirate intercepte le message chiffré sur le réseau, il ne peut pas le lire. Seule la banque, avec sa clé privée RSA, pourra l'ouvrir et traiter ta demande.

2. KEX (Key Exchange - Échange de Clés)
Le KEX est un protocole qui permet à deux personnes qui ne se sont jamais rencontrées de se mettre d'accord sur une clé secrète partagée, tout en discutant sur un canal public (surveillé par un pirate).

Comment ça marche : Contrairement au PKE où l'un envoie une donnée à l'autre, dans un KEX, Alice et Bob collaborent mathématiquement pour construire une clé commune. Chacun choisit un secret mathématique, fait un calcul, l'envoie à l'autre, puis combine le résultat reçu avec son propre secret. Par une magie mathématique, ils aboutissent tous les deux au même résultat final (la clé secrète).

Exemple concret : L'algorithme de Diffie-Hellman (DH ou ECDH). Imagine qu'Alice et Bob veuillent créer une couleur secrète commune sans que personne ne le sache. Ils se mettent d'accord publiquement sur une peinture "jaune". Alice y ajoute en secret du bleu (ça donne du vert), et Bob y ajoute en secret du rouge (ça donne du orange). Ils s'échangent le vert et l'orange (si le pirate intercepte, il a juste du vert et du orange, ce qui ne lui sert à rien car il ne peut pas "dé-mélanger" la peinture). Ensuite, Alice ajoute son bleu secret au orange reçu par Bob, et Bob ajoute son rouge secret au vert reçu par Alice. Les deux obtiennent au final exactement la même couleur marron : c'est leur clé secrète partagée.

3. KEM (Key Encapsulation Mechanism - Mécanisme d'Encapsulation de Clé)
Le KEM est une version moderne et optimisée de l'échange de clés, massivement utilisée dans la nouvelle cryptographie post-quantique. Il permet à un expéditeur de générer simultanément une clé secrète aléatoire et son emballage chiffré (l'encapsulation) pour l'envoyer au destinataire.

Comment ça marche : Alice a une paire de clés (encapsulation/publique et décapsulation/privée). Bob utilise la clé d'encapsulation publique d'Alice avec un algorithme spécial. Cet algorithme fait deux choses en même temps : il génère une clé secrète aléatoire pour Bob, et il fabrique un "coffre-fort mathématique" (le texte chiffré) contenant cette même clé secrète. Bob envoie le coffre-fort à Alice. Alice utilise sa clé de décapsulation privée pour ouvrir le coffre-fort et récupérer la clé secrète.

Exemple concret : Le nouvel algorithme post-quantique ML-KEM (Kyber). Bob veut établir une connexion sécurisée (HTTPS) avec le serveur de Netflix. Bob récupère la clé publique ML-KEM de Netflix. Son navigateur exécute l'algorithme d'encapsulation qui génère une clé symétrique de 256 bits et un paquet chiffré. Bob envoie le paquet chiffré à Netflix. Netflix utilise sa clé privée pour décapsuler le paquet. Maintenant, Bob et Netflix possèdent la même clé de 256 bits et peuvent chiffrer le streaming vidéo avec de l'AES-256 de manière ultra-rapide.Voici un tableau récapitulatif synthétique pour comparer le PKE, le KEX et le

KEM. Il est idéal pour tes fiches de révisions, car il met en évidence les différences de fonctionnement et d'objectif.cloudflare+1

Acronyme	Nom complet	Objectif principal	Fonctionnement	Exemple d'algorithme

Acronyme	Nom complet	Objectif principal	Fonctionnement	Exemple d'algorithme
PKE	**Public Key Encryption** *(Chiffrement à Clé Publique)*	Chiffrer un message ou une donnée de sorte que seul le destinataire puisse le lire [blog.cloudflare].	L'expéditeur prend le message, le chiffre avec la clé publique du destinataire. Le destinataire le déchiffre avec sa clé privée [blog.cloudflare].	RSA, ElGamal.
KEX	**Key Exchange** *(Échange de Clés)*	Créer une clé secrète commune entre deux personnes communiquant sur un réseau non sécurisé [blog.cloudflare].	Méthode collaborative : chacun choisit un secret, fait un calcul mathématique et l'envoie à l'autre. En combinant les résultats, ils obtiennent la même clé [blog.cloudflare].	Diffie-Hellman (DH), ECDH (avec courbes elliptiques).
KEM	**Key Encapsulation Mechanism** *(Mécanisme d'Encapsulation de Clé)*	Générer et transmettre de manière sécurisée une clé de session (pour la cryptographie hybride) cloudflare+1.	L'algorithme génère *simultanément* une clé aléatoire et son "coffre-fort" chiffré. On envoie le coffre au destinataire qui l'ouvre avec sa clé privée pour récupérer la clé cloudflare+1.	ML-KEM (Kyber, standard post-quantique du NIST) [academic.oup] .

Cours : Les Bases de l'Ordinateur

1. Classification et Catégories d'Ordinateurs

Les ordinateurs sont classés selon leur puissance de calcul, leur coût et leur usage. Voici les grandes catégories, de la plus puissante à la plus petite :

Catégorie	Coût estimé	Exemples d'utilisation
Supercalculateurs	~100 M€	Simulations physiques de grande ampleur, prévisions météorologiques.
Ordinateurs centraux (Mainframes)	~1 M€	Systèmes bancaires (transactions massives), blockchains.
Grappes de calcul (Clusters)	~100 k€	Simulations physiques en laboratoire, calculs partagés.
Serveurs	~10 k€	Serveurs réseau, hébergement web et bases de données.
Micro-ordinateurs	~1 k€	Ordinateurs de bureau, PC de jeu, ordinateurs portables (*~150 millions en circulation en 2000*).
Processeurs embarqués	~100 €	Smartphones, tablettes, électronique des véhicules.
Microcontrôleurs	~5 €	Électroménager, domotique, petits projets électroniques.
Puces jetables	~1 €	Cartes à puce (bancaires), puces RFID.

Focus sur les microcontrôleurs (Arduino vs ESP32) :
L'ESP32 est un microcontrôleur très performant mais il est souvent trop puissant (et trop gourmand) pour des projets simples qui ne nécessitent ni Wi-Fi ni Bluetooth. Dans ce cas, un Arduino (utilisant généralement la puce *ATmega328P*) est une meilleure solution : il est beaucoup moins cher et possède une consommation d'énergie beaucoup plus faible que l'ESP32.

2. Un peu d'histoire et les fondamentaux théoriques

L'histoire du calcul a connu plusieurs grandes révolutions technologiques :

- **L'ère manuelle et mécanique (XVIIe - XIXe siècle) :** Moyens de calculs manuels (boulier), puis mécaniques avec des engrenages. L'invention de la *Pascaline* par Blaise Pascal (1623-1662) et la *Machine de Leibniz* marquent les premières calculatrices mécaniques.

- **L'ère des tubes à vide (Années 1940) :** Remplacement de la mécanique par l'électronique avec les tubes à vide. Naissance des premiers vrais ordinateurs

comme le *Colossus* (qui a aidé à casser les codes d'Enigma) et l'*ENIAC*.

- **La révolution des Transistors (Années 1950) :** Inventés par les laboratoires Bell, les transistors en silicium sont beaucoup plus petits, moins chers et beaucoup plus fiables que les fragiles tubes à vide. Exemple : l'*IBM 7094* en 1964.

- **Les Circuits intégrés (1965-1980) :** Regroupement de plusieurs transistors sur une même puce de silicium.

- **Le Parallélisme (Époque moderne) :** Utilisation de plusieurs cœurs et processeurs travaillant en même temps.

Lois et Théories

- **La loi de Moore :** Énoncée en 1965 par Gordon Moore, cette règle stipule que le nombre de transistors sur un microprocesseur double environ tous les 18 à 24 mois à coût constant. Cela explique pourquoi nos ordinateurs deviennent toujours plus petits et plus puissants.wikipedia+1

- **Le problème de l'arrêt (Alan Turing) :** Tu as noté : *"incapable de prédire quand un programme se finit"*. C'est une référence directe au théorème de Turing. Il est mathématiquement impossible de créer un programme universel capable d'analyser n'importe quel autre programme pour deviner s'il va s'arrêter un jour ou s'il va tourner dans une boucle infinie.

3. L'Architecture Matérielle (Hardware)

L'Architecture de Von Neumann

Presque tous les ordinateurs modernes utilisent l'architecture définie par John von Neumann. Elle sépare la machine en plusieurs blocs qui communiquent entre eux

1. L'Unité Centrale de Traitement (Le processeur).

2. La Mémoire (qui stocke les données et les programmes).

3. Les dispositifs d'Entrée / Sortie.

Le Processeur (Micro-architecture)

Le processeur (CPU) est composé de plusieurs éléments clés :info.blaisepascal+1

- **L'ALU (Arithmetic Logic Unit) :** L'Unité Arithmétique et Logique, chargée de faire les calculs mathématiques et de vérifier les conditions logiques.

- **L'Horloge (Le Métronome) :** Elle donne le rythme au processeur (les fameux Gigahertz). À chaque "tic" d'horloge, le processeur effectue une action.

- **L'Unité de contrôle (Le Chef d'orchestre) :** Elle lit les instructions en mémoire, les décode et dirige l'ALU pour exécuter le travail.nicolas.louvet.pages.univ-lyon1+1

- **L'ISA (Instruction Set Architecture) :** C'est le "dictionnaire" du processeur. Il s'agit du jeu d'instructions de base que le processeur est capable de comprendre matériellement.

Les Mémoires

Il existe une hiérarchie stricte dans la mémoire d'un ordinateur (du plus rapide au plus lent) :

- **SRAM (Mémoire Statique) :** Ultra-rapide mais très chère. Utilisée pour les mémoires internes au processeur, appelées les **Registres**, ainsi que les **Caches L1, L2 et L3** (le "LCache").

- **DRAM (Mémoire Dynamique) :** Moins rapide mais beaucoup moins chère. Utilisée pour la mémoire principale de l'ordinateur (la fameuse RAM DDR).

4. Représentation des données et Logique

Unités de mesure (Norme CEI 60027-2)

Il existe une confusion courante entre les multiples décimaux et binaires. La norme CEI 60027-2 a été créée pour clarifier cela :

- Dans le système décimal (base 10) : 1 Gigaoctet (Go) = 1 milliard d'octets (10910^9109).

- Dans le système binaire (base 2 de l'ordinateur) : on utilise les **Gigaoctets (Go)**.

- Par conséquent, 1 milliard d'octets (1 Go) correspond en réalité à environ **0,931 Gio** pour l'ordinateur (car on divise par 102431024^310243).

Les Portes Logiques et l'Addition

L'ordinateur fonctionne en binaire (0 et 1) grâce à des portes logiques (ET, OU, NON).
Pour faire une addition binaire (additionner le bit A et le bit B), on calcule un **Résultat (R)**.

- *Ton équation "Si R = A+B alors C existe entre les 2" :* Le C représente la **Retenue (Carry)**. Si l'addition de A et B dépasse la capacité d'un seul bit (ex: 1 + 1 en binaire fait 10), alors le résultat R est 0, et la retenue C passe au bit supérieur pour continuer le calcul.

5. Architecture logicielle et Programmation

Pour qu'un programme tourne sur l'architecture matérielle (micro-architecture), il faut des instructions.

- **CISC vs RISC :** Ce sont deux philosophies de création de processeurs.
    - *CISC* (Complex Instruction Set Computer) : Le processeur connaît beaucoup d'instructions complexes (ex: architecture x86 utilisée par Intel/AMD sur les PC).
    - *RISC* (Reduced Instruction Set Computer) : Le processeur ne connaît que des instructions simples mais les exécute extrêmement vite (ex: architecture ARM utilisée dans les smartphones).
- **L'Assembleur (ex: x86) :** C'est le langage de programmation de plus bas niveau, le plus proche du matériel. Il parle directement avec l'ISA du processeur.
- **Le langage C :** Il a été inventé pour simplifier la programmation. Il permet d'écrire des programmes plus facilement qu'en Assembleur, puis d'être "compilé" (traduit) en langage machine pour que le processeur l'exécute.

6. L'évolution technologique (Les Transistors)

Le composant de base d'un processeur est le transistor. La technologie a évolué pour consommer moins et être plus rapide :

- **nMOS et pMOS :** Deux types de transistors (type N pour négatif et type P pour positif).
- **CMOS (Complementary Metal Oxide Semiconductor) :** La technologie actuelle révolutionnaire. Elle combine intelligemment un transistor nMOS et un transistor pMOS. Son immense avantage est qu'elle ne consomme de l'énergie que lors des *changements d'état* (quand on passe de 0 à 1), ce qui évite au processeur de surchauffer au repos.

- **Gravure :** La finesse avec laquelle on imprime ces transistors sur le silicium. Aujourd'hui, on grave les circuits en quelques nanomètres (nm), ce qui permet d'en mettre des milliards sur une même puce et de maintenir la loi de Moore en vie.

Architecture Avancée et Performances

1. La gestion de l'énergie et la chaleur

L'un des plus grands défis dans la conception des processeurs est la gestion de la chaleur. Plus un processeur travaille vite, plus il consomme de l'énergie et dissipe de la chaleur.

La Densité de Puissance Dissipée (Formule de base)

Pour comprendre comment un processeur chauffe, on utilise la formule de la puissance dynamique dissipée :
P=C×V2×FP = C \times V^2 \times FP=C×V2×F

- **P : Puissance dissipée (en Watts, qui se transforme en chaleur).**
- **C : Capacité électrique des transistors (Charge à commuter).[fr.wikipedia]**
- **V : Tension d'alimentation (Voltage).[fr.wikipedia]**
- **F : Fréquence d'horloge du processeur (en Hertz, souvent GHz).[fr.wikipedia]**

Le Mur Thermique (Problématiques)

Lorsque l'on veut augmenter la Fréquence (F) pour rendre l'ordinateur plus rapide, la Puissance dissipée (P) augmente dangereusement. Pour éviter que le processeur ne fonde, les ingénieurs doivent jouer sur les autres variables :

- **Baisser P (Puissance de fuite) : Non, car la miniaturisation extrême des composants crée des courants de fuite inévitables (les électrons "sautent" les barrières à cause de la physique quantique).**
- **Baisser V (Tension) : Non, on ne peut pas la baisser indéfiniment à cause du seuil technologique des transistors (ils ont besoin d'un minimum de tension pour distinguer un "0" d'un "1").**
- **Baisser C (Capacité) : C'est la principale solution matérielle. On réduit la taille des transistors (gravure plus fine) pour qu'ils demandent moins d'énergie à**

charger.

Conclusion : Comme on ne peut plus augmenter la fréquence à l'infini à cause de la chaleur, l'industrie a dû trouver d'autres solutions pour gagner en puissance : optimiser l'architecture et passer au parallélisme.

---

2. L'Optimisation des Performances (Le Calcul)

Le temps d'exécution d'un programme dépend de trois facteurs : le nombre d'instructions (NI), le nombre de cycles par instruction (CPI, l'inverse de l'IPC), et le temps d'un cycle (Fréquence).

Comment faire le calcul plus vite ?

- Diminuer le NI (Nombre d'Instructions) ? Oui. C'est le rôle des compilateurs et de l'architecture CISC de créer des instructions complexes qui font plus de choses en une seule ligne de code.[reddit]

- Augmenter l'IPC (Instructions Par Cycle) ? Oui. C'est l'objectif principal des micro-architectures modernes (comme les processeurs d'Apple ou Intel). L'idée est de créer des processeurs capables de lire, décoder et exécuter *plusieurs* instructions en même temps sur un seul "tic" d'horloge (processeurs superscalaires).[reddit]

---

3. Le Parallélisme (SIMD vs SIMT)

Puisqu'on ne peut plus accélérer un seul cœur à l'infini, on multiplie les unités de calcul. Il existe deux grandes architectures parallèles :

- SIMD (Single Instruction, Multiple Data) :
    - *Le principe :* Le processeur envoie une seule instruction (ex: "additionne") qui va s'appliquer simultanément sur plusieurs données différentes alignées dans un vecteur.[studeersnel]
    - *L'utilisation :* Très utilisé dans les CPU classiques (comme les instructions AVX chez Intel) pour traiter du son, de la vidéo ou des calculs mathématiques répétitifs. C'est rigide : toutes les données subissent exactement la même opération au même moment.[studeersnel]

- **SIMT (Single Instruction, Multiple Threads) :**
    - ***Le principe :*** **Modèle massivement utilisé par les Cartes Graphiques (GPU). Une instruction est envoyée, mais elle est exécutée par des milliers de petits "threads" (fils d'exécution) indépendants.[studeersnel]**
    - ***L'avantage :*** **C'est beaucoup plus flexible que le SIMD. Si une donnée nécessite de prendre un chemin différent (un branchement "Si... Alors..."), certains threads peuvent s'adapter (même si cela fait perdre en performance globale).yosefk+1**

---

4. Les Supercalculateurs (Motivations)

Aujourd'hui, l'aboutissement de ces architectures parallèles se trouve dans les Supercalculateurs.

- ***Configuration :*** **Ils ne reposent pas sur de très gros processeurs, mais sur l'assemblage de plusieurs millions de cœurs standards qui calculent en parallèle.**
- ***Consommation :*** **Ils nécessitent une alimentation équivalente à une petite ville (plusieurs Mégawatts - MW) et des systèmes de refroidissement liquides massifs.**
- ***Motivations :*** **À quoi ça sert ? Principalement à remplacer l'expérimentation réelle par la Simulation.**
    - ***Physique :*** **Simulation d'explosions nucléaires (pour éviter les vrais essais), collisions de particules.**
    - ***Météorologie :*** **Modélisation du climat mondial et prédiction des ouragans.**
    - ***Biologie :*** **Modélisation du repliement des protéines pour créer de nouveaux médicaments.**

Fondements de la Sécurité Cryptographique et Attaques

1. Introduction à la Cryptographie Pratique

La cryptographie est la fondation des communications sécurisées modernes. Elle est utilisée partout :

- **Trafic Web :** HTTPS (utilisant les protocoles SSL/TLS).
- **Réseaux sans fil :** Wi-Fi avec WPA2 ou l'ancien WEP (802.11i), réseaux mobiles (GSM), Bluetooth.
- **Protection de contenu (DRM) :** Les DVD (chiffrement CSS) et Blu-ray (AACS).
- **Authentification utilisateur :** Mots de passe, tokens.

Ce que la cryptographie n'est PAS

Comme le rappelle souvent le célèbre cryptologue **Bruce Schneier** : *"La cryptographie n'est pas la solution à tout problème de sécurité."*[schneier]
On ne peut jamais garantir un système sécurisé à 100%. La cryptographie est fiable, sauf si elle est mal implémentée ou si le maillon faible se trouve ailleurs (erreur humaine, faille matérielle, etc.). La complexité est l'ennemi de la sécurité.schneier+2

Note historique : Les Prix Turing (l'équivalent du Prix Nobel en informatique) ont souvent récompensé des cryptographes de génie : R. Rivest, A. Shamir, L. Adleman (inventeurs du RSA), S. Goldwasser, ou encore W. Diffie et M. Hellman (échange de clés).

Principe de Kerckhoffs : Sécuriser la Clé vs Obfusquer l'Algorithme

Il ne faut jamais baser la sécurité d'un système sur le secret de son algorithme de chiffrement (obfuscation ou *security by obscurity*). Un algorithme de chiffrement robuste doit être public et connu de tous. **Seule la clé doit être gardée secrète.**

---

2. Syntaxe et Sécurité d'un Schéma de Chiffrement

Un schéma de chiffrement symétrique (Private Key Encryption) se compose obligatoirement de 3 algorithmes :

1. **Génération de clé (Key Generation) :** Crée une clé aléatoire.
2. **Chiffrement (Encryption) :** Transforme le message clair (*plaintext*) en message chiffré (*ciphertext*) grâce à la clé.

3. **Déchiffrement (Decryption) :** Opération inverse.
   Condition de base (Correctness requirement) : Si on chiffre un message avec une clé, puis qu'on le déchiffre avec cette même clé, on doit toujours retomber exactement sur le message clair de départ.

Le secret parfait (Perfect Secrecy) et l'OTP

Le **One Time Pad (OTP)** ou Chiffre de Vernam est le seul système offrant un **secret parfait**. Le secret parfait signifie que le message chiffré ne révèle *absolument aucune information* sur le message clair à un attaquant (même s'il a une puissance de calcul infinie). La probabilité de deviner le message reste la même, avec ou sans le *ciphertext*.

Sécurité face à un espion (Eavesdropper) et notion "Indistinguishable"

Dans la réalité, l'OTP n'est pas pratique. On utilise donc des algorithmes modernes et on définit la sécurité de manière mathématique.
Un schéma de chiffrement est considéré comme sécurisé (on parle d'**Indistinguishable encryption in the presence of an eavesdropper**) si :

- Face à un adversaire "A" qui est un algorithme **PPT (Probabilistic Polynomial-Time)**, c'est-à-dire un attaquant réaliste limité par la puissance de calcul des ordinateurs actuels...

- ...La probabilité que cet attaquant réussisse à distinguer le chiffrement de deux messages différents s'approche de 0.[ktln2]

- Mathématiquement, l'avantage de l'attaquant doit être inférieur à une **fonction négligeable** (*negl function*). Une fonction négligeable décroît tellement vite vers 0 que le risque d'attaque devient statistiquement impossible dans le monde réel.[ktln2]

---

3. Les Attaques dans le Monde Réel

L'échec du WEP (802.11b)

Le WEP était l'ancien système de sécurité du Wi-Fi. Il chiffrait les données avec un flux continu. Pour éviter de chiffrer deux paquets avec la même clé, le WEP ajoutait un vecteur d'initialisation (IV) de **24 bits**.

- **Le problème :** Avec seulement 24 bits, il n'y a que 2242^{24}224 (soit environ 16 millions) de combinaisons possibles d'IV. Sur un réseau Wi-Fi très actif, ces 16

millions de combinaisons s'épuisent très vite. Le routeur finissait par réutiliser le même IV.

- **Pire encore :** Sur certaines cartes Wi-Fi, l'IV se remettait simplement à zéro à chaque redémarrage (power cycle). En interceptant deux messages chiffrés avec le même IV, les pirates pouvaient facilement annuler le chiffrement (via un XOR) et retrouver la clé.

L'Attaque à Texte Clair Choisi (CPA - Chosen Plaintext Attack)

Une attaque **CPA** est un modèle d'attaque cryptographique où l'attaquant a la capacité de choisir des textes clairs de son choix et d'obtenir leurs versions chiffrées correspondantes par le système. Son but est d'étudier comment la machine réagit pour en déduire la clé secrète.

L'histoire de la Bataille de Midway (Seconde Guerre mondiale) :
C'est l'un des exemples historiques les plus célèbres de cryptanalyse s'apparentant à de l'injection de texte clair.
En 1942, les cryptanalystes de l'US Navy interceptaient les messages japonais chiffrés. Ils avaient remarqué que les Japonais préparaient une attaque massive contre une cible nommée **"AF"**.doncio.navy+1

- Les cryptanalystes de Pearl Harbor pensaient que "AF" signifiait "l'île de Midway", mais l'état-major américain n'y croyait pas et pensait que c'était Hawaï ou la côte ouest.[nsa]

- Pour en avoir le cœur net, les Américains ont organisé un piège (l'injection d'un texte clair connu) : ils ont ordonné à la garnison de Midway d'envoyer un faux message radio non chiffré, se plaignant d'une **panne grave du système de désalinisation d'eau (Water shortage)**.doncio.navy+1

- Peu de temps après, les Américains ont intercepté un message chiffré japonais disant : *"AF manque d'eau douce"*.nsa+1

- Le doute était levé : "AF" était bien Midway. Grâce à cette ruse, les Américains ont pu préparer une embuscade et gagner la bataille de Midway, un tournant décisif de la guerre du Pacifique.[nsa]

Cryptographie Avancée et Modes d'Opération

1. Concept de PRF (Pseudo-Random Function)

En cryptographie, on ne peut pas générer du vrai hasard (aléatoire pur) de manière purement logicielle. On utilise donc des PRF (Pseudorandom Functions).

- **Définition : Une fonction pseudo-aléatoire est un algorithme déterministe qui prend une clé secrète et une entrée, et retourne une sortie qui "ressemble" à du vrai hasard.[faculty.cc.gatech]**

- **Sécurité du PRF : La sécurité d'une PRF repose sur l'indistinguabilité. Face à un adversaire limité en puissance de calcul (PPT - *Probabilistic Polynomial-Time*), il doit être impossible de faire la différence entre la sortie de la PRF et une suite de bits générée par une fonction véritablement aléatoire.intensecrypto+1**

Lien entre PRF et sécurité CPA

Pour qu'un schéma de chiffrement soit CPA-Secure (sécurisé contre les attaques à texte clair choisi), il ne doit jamais être déterministe. Si tu chiffres deux fois le même message avec la même clé, le résultat doit être différent à chaque fois.
C'est là que la PRF intervient : en combinant la clé secrète avec une valeur aléatoire (comme un vecteur d'initialisation - IV), la PRF permet de générer un chiffrement unique à chaque fois, garantissant ainsi la sécurité CPA.[intensecrypto]

---

2. Le mode ECB (Electronic Codebook) et sa vulnérabilité

L'AES chiffre les données par blocs (ex: blocs de 16 octets). Le mode ECB est la façon la plus naïve de chiffrer : il prend le bloc 1, le chiffre, puis prend le bloc 2, le chiffre de la même façon avec la même clé, et ainsi de suite.

L'attaque (ou faille) en mode ECB

Le mode ECB n'est pas CPA-secure car il est déterministe. Si deux blocs de texte clair sont identiques, ils produiront exactement le même bloc chiffré.tonybox+1

- **L'exemple célèbre du Pingouin (Tux) : Si tu chiffres l'image du pingouin Linux en mode ECB, la forme du pingouin reste parfaitement visible sur l'image chiffrée. L'image est brouillée (couleurs altérées), mais comme le fond blanc est constitué de blocs identiques, ils sont tous chiffrés avec la même valeur, révélant la structure géométrique de l'image.linkedin+1**
  Règle d'or : Ne jamais utiliser le mode ECB.

---

3. Les autres modes (CBC, OFB, CTR) et le problème de l'IV

Pour résoudre le problème de l'ECB, on utilise d'autres modes qui injectent un IV (Initialization Vector). L'IV permet de s'assurer que même si le premier bloc de texte est identique, le résultat chiffré sera différent.[cwe.mitre]

- **CBC (Cipher Block Chaining) : Le bloc clair est "XORé" avec le bloc chiffré précédent avant d'être lui-même chiffré. Pour le tout premier bloc, on le XOR avec l'IV.[cwe.mitre]**

- **OFB (Output Feedback) : Transforme l'AES en chiffrement par flux. On chiffre l'IV en boucle pour créer un flux pseudo-aléatoire, qu'on XOR ensuite avec le message clair.[intensecrypto]**

- **CTR (Counter Mode) : Le meilleur mode actuel. On chiffre un "compteur" (qui combine un IV (nonce) et un nombre qui s'incrémente) pour créer un flux pseudo-aléatoire, XORé ensuite avec le message.**

L'attaque de l'IV prédictible

Pour que le mode CBC soit sécurisé contre les attaques CPA, l'IV doit être strictement aléatoire et imprévisible pour l'attaquant.[cwe.mitre]

- **Si l'IV n'est pas aléatoire (ex: un IV fixe à "0000", ou un IV qui s'incrémente logiquement : 1, puis 2, puis 3), le chiffrement devient vulnérable.[cwe.mitre]**

- **Un attaquant (CPA) qui sait quel sera le prochain IV peut préparer une attaque ciblée pour déduire le contenu du premier bloc de texte clair.[cwe.mitre]**

---

4. L'attaque par Padding Oracle (Oracle de Remplissage)

Cette attaque cible spécifiquement le mode CBC lorsqu'il est utilisé sur un serveur communicant (comme le vieux HTTPS/TLS).

Le contexte

En CBC, si le message clair n'est pas un multiple exact de la taille du bloc (16 octets), il faut ajouter du "bourrage" (padding) à la fin (ex: des octets `03 03 03` s'il manque 3 octets).
Lorsqu'un serveur déchiffre un message, il vérifie le padding. Si le padding est correct, il dit "OK" (ou traite la requête). Si le padding est incorrect, il renvoie une erreur spécifique (ex: "Padding Error"). Le serveur agit donc comme un Oracle qui dit "Vrai" ou "Faux".

L'attaque (En modifiant le 1er octet)

- **L'attaquant intercepte un message chiffré.**
- **L'attaquant ne connaît pas la clé, mais il peut modifier le message chiffré. Il va envoyer des centaines de faux messages au serveur en modifiant bit par bit le bloc chiffré précédent (ou en modifiant le 1er byte de l'IV pour attaquer le premier bloc clair).**
- **L'attaquant observe la réaction du serveur. Si le serveur renvoie "Erreur de padding", l'attaquant sait que sa modification était mauvaise.**
- **S'il modifie l'IV et que le serveur renvoie une erreur différente (ou accepte le message silencieusement), l'attaquant sait qu'il est tombé juste sur un padding valide.**
- **Par une simple déduction mathématique (grâce au XOR de l'IV en CBC), l'attaquant peut retrouver le contenu en clair de ce byte. En répétant l'opération, il peut déchiffrer l'intégralité du message sans jamais posséder la clé.**

Chronologie du DES (Data Encryption Standard)

Les Origines (Années 1970)

- **1973 : Le gouvernement américain, via le NBS (National Bureau of Standards, aujourd'hui NIST), lance un appel d'offres public pour trouver un algorithme de chiffrement standard afin de protéger les données non classifiées (notamment pour les banques).wikipedia+1**

- **1974 : L'équipe de cryptographes d'IBM (dont Horst Feistel) propose un algorithme basé sur leur précédent chiffrement appelé *Lucifer*.onlinehashcrack+1**

- **1975 : L'algorithme d'IBM est publié pour évaluation publique. C'est à ce moment-là que la NSA intervient pour modifier légèrement l'algorithme, notamment en réduisant la taille de la clé de 128 bits à 56 bits et en modifiant certaines tables de substitution (S-boxes).mtlc+1**

L'Adoption comme Standard

- **Novembre 1976 : L'algorithme est officiellement approuvé comme standard fédéral.[en.wikipedia]**

- **Janvier 1977 : Le DES est officiellement publié sous la norme FIPS PUB 46. Il devient le tout premier algorithme de chiffrement validé par le gouvernement américain pour un usage civil et commercial (il utilise des blocs de 64 bits et une clé effective de 56 bits).theknowledgeacademy+2**

- **Juin 1977 : Les cryptographes Diffie et Hellman (les pères de l'échange de clés asymétrique) critiquent publiquement le DES. Ils préviennent déjà que la clé de 56 bits est trop courte et pourrait être cassée par force brute par un ordinateur très puissant.[en.wikipedia]**

Le Déclin et les Failles (Années 1990)

- **1992 : Les mathématiciens Biham et Shamir publient une technique appelée *cryptanalyse différentielle* (première attaque théorique plus rapide que la force brute contre le DES), révélant que les modifications secrètes faites par la NSA en 1975 servaient en réalité à protéger le DES contre cette attaque.[en.wikipedia]**

- **Juin 1997 : Le projet "DESCHALL" réussit pour la toute première fois à déchiffrer un message DES en public.[en.wikipedia]**

- **Juillet 1998 : L'association EFF (Electronic Frontier Foundation) construit une machine spéciale pour seulement 250 000 $, appelée "Deep Crack". Elle parvient à casser une clé DES par force brute en seulement 56**

heures.[en.wikipedia]

- Janvier 1999 : En combinant la machine Deep Crack et un réseau d'ordinateurs distribués, une clé DES est cassée en 22 heures et 15 minutes. Le standard est définitivement obsolète.[en.wikipedia]

La Fin du DES

- Octobre 1999 : Face à ces vulnérabilités, le gouvernement américain publie une révision forçant l'utilisation du Triple DES (3DES), qui applique l'algorithme trois fois de suite avec des clés différentes pour augmenter la sécurité (clé de 168 bits). Le DES simple est relégué aux vieux systèmes.wikipedia+1

- Novembre 2001 : Le standard DES est officiellement remplacé par le nouveau standard AES (Advanced Encryption Standard).ibm+1

- Mai 2005 : Le NIST retire officiellement et définitivement la certification du DES (FIPS 46-3).[en.wikipedia]

Cryptographie Symétrique et Fondements Théoriques

1. Block Ciphers vs Stream Ciphers

Il existe deux grandes familles d'algorithmes de chiffrement symétrique pour transformer le texte clair en texte chiffré : les algorithmes par blocs (Block ciphers) et les algorithmes par flux (Stream ciphers).[thesslstore]

Block Ciphers (Chiffrement par blocs)

- Le principe : L'algorithme prend une taille fixe de données (un bloc) à la fois, généralement 64 ou 128 bits (comme AES). Il mixe ce bloc entier avec la clé via des opérations complexes de confusion et de diffusion.geeksforgeeks+1

- Le problème de taille (Padding) : Si le message (ou le dernier bout du message) ne fait pas exactement la taille requise par le bloc, l'algorithme ne peut pas fonctionner. On doit donc utiliser un algorithme de *Padding* (Bourrage) pour remplir l'espace vide avec des caractères

spécifiques.engineering.purdue+1

- Vulnérabilité (Message Tampering) : En fonction du mode d'opération (comme le mode ECB ou CBC), un attaquant peut manipuler (tampering) un bloc de ciphertext spécifique. Comme le bloc a une taille fixe, il est parfois possible de modifier un octet précis, ce qui altérera un bloc précis lors du déchiffrement sans que le destinataire ne s'en rende compte (d'où l'existence des attaques par *Padding Oracle*).[thesslstore]

Stream Ciphers (Chiffrement par flux)

- Le principe : Au lieu de prendre un bloc, l'algorithme chiffre le message *bit par bit*. Il utilise la clé pour générer un flux continu de bits pseudo-aléatoires (Keystream) qui est ensuite combiné au message clair via une simple opération XOR.geeksforgeeks+1

- Avantages : Très rapide, consomme peu de ressources (idéal pour le matériel embarqué), et il n'y a absolument aucun besoin de padding.wizardforcel.gitbooks+1

- Vulnérabilité : Si la même clé/IV est réutilisée (comme dans la faille WEP étudiée précédemment), le système est cassé. De plus, ils sont très vulnérables aux attaques de manipulation de bits (*bit-flipping attacks*) : si un pirate inverse un "1" en "0" dans le chiffré, le bit correspondant dans le message clair sera exactement inversé sans créer d'erreur de bloc.[thesslstore]

---

2. Les Fonctions à Sens Unique (One Way Functions - OWF)

Les fonctions à sens unique sont la brique fondatrice absolue de la cryptographie moderne.[cs.jhu]

- Définition : C'est une fonction mathématique facile et rapide à calculer dans un sens, mais (actuellement) impossible à inverser dans un temps raisonnable.haifa+1

Exemples mathématiques

Les algorithmes asymétriques reposent entièrement sur ces problèmes mathématiques insolubles à l'envers :

- **La Multiplication et la Factorisation (Base du RSA) : Il est très facile pour un ordinateur de multiplier deux nombres premiers gigantesques pour obtenir un très grand résultat (ex: p×q=Np \times q = Np×q=N). Mais si on te donne uniquement NNN, il est informatiquement impossible de retrouver ppp et qqq (factorisation en nombres premiers).**

- **Le Carré Modulaire (Modular Squaring, basé sur le problème de Rabin) : Il est facile de calculer le carré d'un nombre modulo NNN (c'est-à-dire x2(modN)x^2 \pmod Nx2(modN)). Mais si on te donne le résultat et NNN, trouver la racine carrée (retrouver xxx) est aussi difficile que de factoriser NNN.**

---

3. Constructions Théoriques d'Objets Pseudo-aléatoires

En théorie de la cryptographie, on s'appuie sur une hiérarchie stricte d'hypothèses pour construire des systèmes sécurisés. Tout découle de l'existence présumée des fonctions à sens unique (OWF).[cs.uni-paderborn]

- **L'hypothèse fondatrice : On part du principe que les OWF existent.**

- **Le Générateur (PRG - Pseudorandom Generator) : À partir d'une OWF, on peut mathématiquement prouver qu'il est possible de construire un PRG, capable d'étirer une petite clé secrète (seed) en une très longue suite de bits pseudo-aléatoires.[cs.uni-paderborn]**

- **La Fonction (PRF - Pseudorandom Function) : Si on possède un PRG, le théorème de Goldreich-Goldwasser-Micali prouve qu'on peut construire une PRF sécurisée (utilisée dans l'AES, par exemple).intensecrypto+1**

- **Conclusion : C'est ce qui permet d'affirmer mathématiquement que nos chiffrements symétriques sont sûrs... tant que personne ne prouve que P=NPP = NPP=NP et n'invente un algorithme pour inverser les OWF.**

---

4. Intégrité et Authentification (Message Authentication Code - MAC)

Jusqu'ici, le chiffrement garantissait la *confidentialité*. Mais un chiffrement seul ne protège pas contre la manipulation (Message Tampering).

Le problème

Un attaquant (*Mallory*) intercepte ton message chiffré destiné à ta banque : "Transférer 100€ à Alice". Mallory modifie quelques bits au hasard et le renvoie. La banque déchiffre le message modifié qui devient : "Transférer 900€ à Mallory". Le chiffrement n'a rien empêché, car la banque ne peut pas vérifier l'intégrité du message.

La solution : Authentification

Pour empêcher l'altération, l'émetteur et le destinataire doivent partager une clé secrète pour générer un MAC (Message Authentication Code), qui fonctionne comme une empreinte numérique sécurisée.

1. **L'émetteur calcule un "Tag" (une balise) mathématique basé sur le message ET la clé secrète.**

2. **Il envoie le message + le Tag.**

3. **Le destinataire recalcule le Tag de son côté. Si un seul bit du message a été modifié en route (par un problème de réseau ou un attaquant), les Tags ne correspondront plus. Le destinataire rejettera immédiatement le message.**

- ***Exemple de schémas modernes (Chiffrement Authentifié) :* AES-GCM (Galois/Counter Mode). Il chiffre le message (confidentialité) et crée un tag (intégrité/authentification) en même temps.**

Intégrité, Authentification et Cryptographie Asymétrique

1. La syntaxe et la sécurité du MAC (Message Authentication Code)

Un MAC sert à garantir l'intégrité et l'authenticité d'un message partagé entre deux parties possédant la même clé secrète.

Syntaxe standard d'un schéma MAC

Il se compose de trois algorithmes de base :

1. Gen (KeyGen) : Génère une clé secrète kkk.

2. Mac : Prend la clé kkk et un message mmm, puis génère une balise (Tag) ttt.

3. Verify : Prend kkk, mmm et ttt. Renvoie "Vrai" (1) si la balise correspond bien au message, et "Faux" (0) sinon.

La sécurité : EUF-CMA

Le standard absolu de sécurité pour un MAC (ou une signature numérique) s'appelle EUF-CMA (*Existential Unforgeability under adaptive Chosen Message Attack*).pmc.ncbi.nlm.nih+1

- L'attaque (CMA) : L'attaquant possède un "Oracle" à qui il peut demander de générer des MACs pour des millions de messages de son choix.cryptographyengineering+1

- La victoire (Existential Forgery) : Pour que le MAC soit considéré comme cassé, l'attaquant doit réussir à générer un Tag valide pour un nouveau message (qu'il n'a jamais demandé à l'Oracle auparavant). S'il n'y arrive pas, le MAC est "EUF-CMA secure".[pmc.ncbi.nlm.nih]

L'attaque par Rejeu (Replay Attack)

Le MAC garantit *qui* a envoyé le message, mais pas *quand*.

- L'attaque : Un attaquant intercepte un message valide avec son MAC (ex: "Transférer 100€ à Bob", Tag: `XYZ`). L'attaquant ne peut pas modifier le message, mais il peut le renvoyer 10 fois à la banque. La banque vérifiera le MAC, verra qu'il est valide, et exécutera l'ordre 10 fois.

- La parade : On intègre des numéros de séquence (`Message n°1`, `Message n°2`) ou des horodatages (timestamps) à l'intérieur du message avant de calculer le MAC. Si la banque reçoit deux fois le "Message n°1", elle rejette le second.

---

2. Combiner Chiffrement et Authentification

Historiquement, on utilisait un algorithme pour le chiffrement (confidentialité) et un autre pour le MAC (intégrité). La grande question était : dans quel ordre faut-il les appliquer ?[daemonology]

Les 3 approches historiques

1. **Encrypt-and-MAC (ex: SSH) : On chiffre le message clair mmm, et *en parallèle*, on calcule le MAC sur le message clair mmm. On envoie le Chiffré + le MAC. Problème : Comme le MAC est calculé sur le texte clair, il pourrait fuiter des informations mathématiques sur le contenu du message.[daemonology]**

2. **MAC-then-Encrypt (ex: SSL/TLS historique) : On calcule le MAC sur le texte clair, on l'attache à la fin du message, puis on chiffre *le tout*. Problème : C'est ce qui a rendu les anciennes versions de SSL vulnérables aux fameuses attaques par *Padding Oracle* (vues précédemment).[daemonology]**

3. **Encrypt-then-MAC (ex: IPsec) : On chiffre d'abord le texte clair. Ensuite, on calcule le MAC sur le texte chiffré. On envoie le Chiffré + le MAC. Théorie : C'est la méthode la plus sécurisée. Le destinataire vérifie le MAC avant même d'essayer de déchiffrer. Si le MAC est faux, il rejette le paquet immédiatement, annulant toute attaque de type Padding Oracle.[youtube][daemonology]**

La norme moderne : L'AEAD (Authenticated Encryption with Associated Data)

Aujourd'hui, pour éviter les erreurs d'implémentation, on n'utilise plus des algorithmes séparés. On utilise des modes AEAD qui font le chiffrement et le MAC en une seule passe optimisée :[mailarchive.ietf]

- **GCM (Galois/Counter Mode) : Le standard absolu sur Internet aujourd'hui. Il combine le mode CTR pour le chiffrement et la fonction mathématique de Galois pour le MAC. Il est très rapide et parallélisable.[youtube][mailarchive.ietf]**

- **CCM : Combine le mode CTR et le mode CBC-MAC. Très utilisé dans le sans-fil (WPA2, Bluetooth).**

- **EAX : Conçu pour pallier certains défauts théoriques du CCM.[mailarchive.ietf]**

- **OCB : Le mode le plus rapide de tous, mais il a été longtemps boudé car il était protégé par des brevets militaires/industriels.[mailarchive.ietf]**

---

3. Fondements de la Cryptographie Asymétrique (Clé Publique)

Contrairement à la cryptographie symétrique (1 clé partagée), la cryptographie asymétrique utilise une paire de clés (Clé publique pour chiffrer/vérifier, Clé privée pour déchiffrer/signer).

Les Forces de l'Asymétrique

1. **Distribution des clés : Plus besoin de se rencontrer en secret pour échanger une clé. La clé publique peut être affichée partout.**

2. **Non-répudiation (Signatures) : Seul le détenteur de la clé privée peut générer une signature. Il ne peut donc pas nier avoir signé un document.**

Concepts Théoriques

- **Trapdoor Permutations (Permutation à trappe) : C'est la base du RSA. C'est une fonction mathématique facile à calculer dans un sens (ex: chiffrer), mais informatiquement impossible à inverser (déchiffrer)... sauf si l'on possède une "trappe secrète" (la clé privée, qui permet d'inverser la fonction instantanément).**

- **L'hypothèse DDH (Decisional Diffie-Hellman) : C'est un problème mathématique basé sur les courbes elliptiques et les logarithmes discrets. L'hypothèse postule que si un attaquant possède gxg^xgx et gyg^ygy, il est incapable de faire la différence entre gxyg^{xy}gxy (le vrai secret partagé) et une valeur totalement aléatoire gzg^zgz. C'est la base de la sécurité des échanges de clés.**

- **Sécurité CCA2 (Adaptive Chosen Ciphertext Attack) : C'est le standard de sécurité maximum pour le chiffrement asymétrique. Dans ce modèle, l'attaquant possède un "oracle de déchiffrement". Il peut lui envoyer n'importe quel texte chiffré pour obtenir sa version claire, sauf le "Texte Chiffré Cible" qu'il essaie de casser. Un système IND-CCA2 résiste à ce niveau de manipulation.**

Zero-Knowledge Proofs (Preuves à Divulgation Nulle)

Un concept fascinant de la cryptographie asymétrique. Il s'agit d'un protocole permettant à "Prouveur" de prouver mathématiquement à un "Vérificateur" qu'il possède une information secrète (comme un mot de passe ou une clé), sans jamais révéler cette information. (Très utilisé aujourd'hui dans l'authentification et les blockchains privées).

Cours : Échange de Clés et Infrastructures de Confiance (PKI)

1. L'Échange de clés Diffie-Hellman et les Mathématiques

L'algorithme de Diffie-Hellman est une méthode fondatrice qui a révolutionné la cryptographie dans les années 70. Il permet à deux personnes de s'accorder sur une clé secrète commune en utilisant un canal de communication non sécurisé (surveillé par un pirate), sans jamais envoyer cette clé.wikipedia+1

Les bases mathématiques

La sécurité de Diffie-Hellman repose sur des mathématiques modulaires très difficiles à inverser, notamment le calcul du logarithme discret dans un corps fini.[en.wikipedia]
Dans ce contexte, on utilise souvent un théorème célèbre pour optimiser les calculs ou prouver des propriétés : le Théorème des Restes Chinois (Chinese Remainder Theorem - CRT).

- **Le principe du CRT : C'est un théorème d'arithmétique modulaire qui affirme que si l'on connaît les restes de la division d'un nombre par plusieurs entiers premiers entre eux, alors il est possible de déterminer ce nombre de manière unique modulo le produit de ces entiers. Il est notamment massivement utilisé pour accélérer le déchiffrement et les signatures de l'algorithme RSA.[ebooks.inflibnet.ac]**

---

2. Infrastructure à Clés Publiques (PKI)

Une PKI (Public Key Infrastructure) est un ensemble de composants (logiciels, matériels, politiques) qui permet de distribuer et de vérifier les clés publiques de manière sécurisée via des Certificats Numériques (la "carte d'identité" d'une clé publique).ravenswoodtechnology+1

Le problème : Comment faire confiance à une clé publique ?

Si un serveur te donne sa clé publique en disant "Je suis ta banque", comment être sûr qu'un pirate ne s'est pas fait passer pour la banque ? La solution est l'utilisation d'une CA (Certificate Authority / Autorité de Certification).[reddit]

- Seul CA (Modèle simple) : Dans un réseau fermé (comme une petite entreprise), on peut avoir une seule CA racine. Tous les appareils de l'entreprise font une confiance absolue à cette machine. Elle signe directement le certificat de chaque utilisateur.[gradenegger]

- Multiple CA (Modèle hiérarchique) : Sur Internet, on utilise plusieurs niveaux. C'est plus sécurisé et plus scalable.[gradenegger]

- Délégation et Chaines de certificats (Certificate Chains) : La *Root CA* (Autorité Racine) est hyper sécurisée et souvent déconnectée d'Internet. Elle signe les certificats de *CA Intermédiaires* (Intermediate CAs). Ces CA intermédiaires délèguent ensuite la confiance et signent les certificats des serveurs finaux (End-entity certificates, ex: un site web). Ton navigateur vérifie la chaîne de bas en haut : du site web, vers la CA intermédiaire, puis vers la Root CA incluse dans ton navigateur.[ravenswoodtechnology]

L'alternative : Le Web of Trust (PGP)

Contrairement à la PKI (où l'on fait confiance à de grandes entreprises centralisées comme DigiCert ou Verisign), le système PGP (Pretty Good Privacy) utilise la "Toile de confiance" (Web of Trust).[reddit]

- Principe : Il n'y a pas d'autorité centrale. Les utilisateurs signent numériquement les clés publiques des autres utilisateurs. Si Alice fait confiance à Bob, et que Bob a signé la clé de Charlie, alors Alice peut faire confiance à la clé de Charlie. C'est un réseau de confiance décentralisé.[reddit]

---

3. Recommandations Générales en Cryptographie (Les Bonnes Pratiques)

Pour clore ce cours, voici les règles d'or absolues pour tout expert en cybersécurité face à la cryptographie :

1. Utiliser UNIQUEMENT des algorithmes et protocoles standardisés : Ne jamais essayer d'inventer son propre algorithme de chiffrement ("Do Not Roll Your Own Crypto"). Les algorithmes validés (AES, RSA, ECC, SHA) ont été testés par des milliers de mathématiciens pendant des décennies.

2. **Pas de primitives : Les développeurs ne doivent pas manipuler directement des "primitives" cryptographiques (comme le bloc AES brut). Il faut utiliser des bibliothèques de haut niveau (libsodium, Tink) qui gèrent automatiquement et correctement les modes d'opération (GCM), les IV aléatoires et le padding.**

3. **Pas de sécurité dans l'obscurité (Security by Obscurity) : Comme vu précédemment (Principe de Kerckhoffs), cacher le code source de l'algorithme ne sert à rien. Un système sécurisé doit rester sécurisé même si le pirate connaît parfaitement le fonctionnement du code. Seule la clé doit rester secrète.**

Signature Numérique et Certificats

1. Confidentialité vs Intégrité et Authentification

- **La Confidentialité : Si Alice et Bob partagent une *clé secrète* (chiffrement symétrique), ils peuvent chiffrer et déchiffrer un message. Un attaquant qui intercepte le message ne pourra pas le lire.[cnil]**

- **L'Intégrité et l'Authentification : Le chiffrement ne suffit pas toujours. Bob a besoin d'être absolument certain que le document qu'il a reçu provient bien d'Alice (Authentification) et qu'il n'a pas été modifié par l'attaquant en cours de route (Intégrité). C'est ici qu'intervient la signature numérique (qui utilise la cryptographie asymétrique).sectigo+1**

---

2. Le processus de la Signature Numérique (par Alice)

Contrairement au chiffrement où l'on utilise la clé publique du destinataire, pour signer, on utilise sa propre clé privée.esignglobal+1
Voici le schéma exact étape par étape (comme tu l'as noté) :

1. **La Donnée (Le document) : Alice écrit son document.**

2. **Le Hachage : Le document passe dans une fonction de hachage (ex: SHA-256).[esignglobal]**

3. **L'Empreinte (Hash) : La fonction de hachage produit une empreinte numérique unique de taille fixe.[esignglobal]**

4. Chiffrement avec la Clé Privée : Alice chiffre *uniquement cette empreinte* (et non tout le document) en utilisant sa clé privée. L'utilisation de la clé privée est la preuve mathématique que c'est bien Alice qui a fait l'action (Non-répudiation).eitca+2

5. La Signature : L'empreinte chiffrée devient la "Signature Numérique".eitca+1

6. La Donnée Signée : Alice envoie à Bob le document original en clair, accompagné de la signature numérique.[fr.eitca]

---

3. Le processus de Vérification de la Signature (par Bob)

Lorsque Bob reçoit le document signé, il doit prouver que la signature correspond bien au document.

1. Séparation : Bob sépare le document reçu de la signature.

2. Déchiffrement de la signature : Bob utilise la clé publique d'Alice pour déchiffrer la signature. S'il y parvient, cela prouve indiscutablement que le message a été signé par la clé privée d'Alice (Authentification). Le déchiffrement lui révèle l'*Empreinte A* (le hash d'origine).esignglobal+1

3. Nouveau Hachage : Bob prend le document reçu et le passe lui-même dans la même fonction de hachage qu'Alice. Il obtient une nouvelle empreinte, l'*Empreinte B*.eitca+1

4. La Comparaison : Bob compare l'*Empreinte A* et l'*Empreinte B*.

    - Si A = B : Le document est authentique et n'a pas été altéré d'un seul bit (Intégrité confirmée).esignglobal+1

    - Si A ≠\neq= B : Un attaquant a modifié le document pendant le transfert. La signature est déclarée invalide.[sectigo]

---

4. Le Rôle du Certificat Numérique

Pour que l'étape de vérification fonctionne, Bob doit utiliser la clé publique d'Alice. Mais comment Bob peut-il être sûr que la clé publique qu'il possède appartient *vraiment* à Alice et non à un pirate (qui se ferait passer pour elle) ?

C'est le rôle du Certificat Numérique (ex: format X.509).chambersign+1

- **Un certificat est un document numérique (une carte d'identité) délivré par un Tiers de Confiance appelé Autorité de Certification (CA).[chambersign]**
- **Ce certificat contient : le nom d'Alice, sa fameuse clé publique, les dates de validité, et surtout la propre signature numérique de la CA.yousign+1**
- **En liant mathématiquement l'identité d'Alice à sa clé publique, le certificat empêche toute usurpation d'identité.[yousign]**

RSA vs ECC : Comparatif de taille et de sécurité

Avant de manipuler les clés, il faut comprendre pourquoi on utilise l'un ou l'autre :

- **RSA (Rivest-Shamir-Adleman) repose sur la difficulté de factoriser de grands nombres premiers. Pour être sécurisé aujourd'hui (et résister aux attaques par force brute ou algorithme de Shor classique), une clé RSA doit faire 2048 bits ou 4096 bits.**
- **ECC (Elliptic Curve Cryptography) repose sur le problème du logarithme discret sur des courbes algébriques. C'est mathématiquement beaucoup plus complexe à inverser que le RSA. Ainsi, une clé ECC de seulement 256 bits offre la même sécurité qu'une énorme clé RSA de 3072 bits !**
- **Bilan : ECC est plus léger, plus rapide et consomme moins d'énergie (idéal pour les smartphones ou l'IoT), tandis que RSA reste largement supporté par les anciens systèmes.**

Partie 1 : Manipulation des Clés (RSA et ECC)

1. Générer une clé privée RSA

bash

```
openssl genrsa -out private_rsa.pem 2048
```

- **Pourquoi : Crée le fichier de base (le "coffre-fort" mathématique) qui contiendra les paramètres privés et publics.[stackoverflow]**

- **Vulnérabilité : Si tu choisis une taille de 1024 bits, la clé est vulnérable à la force brute. Il faut toujours utiliser 2048 ou 4096 bits.[stackoverflow]**

2. Afficher la clé privée RSA et ses informations

bash

```
openssl rsa -in private_rsa.pem -text -noout
```

- **Pourquoi : Affiche la structure mathématique interne (le "Modulus" commun, les nombres premiers `p` et `q`, et les exposants).[stackoverflow]**

3. Extraire la clé publique RSA

bash

```
openssl rsa -in private_rsa.pem -pubout -out public_rsa.pem
```

- **Pourquoi : Une clé privée RSA contient mathématiquement la clé publique. Cette commande "découpe" la partie publique pour que tu puisses l'envoyer à tes correspondants sans révéler ton secret.[stackoverflow]**

4. Afficher la clé publique RSA

bash

```
openssl rsa -in public_rsa.pem -pubin -text -noout
```

- **Pourquoi : Affiche uniquement le Modulus (`n`) et l'exposant public (`e`). C'est ce que tout le monde peut voir.[stackoverflow]**

---

5. Générer une clé privée ECC (Courbes elliptiques)

bash

```
openssl ecparam -genkey -name prime256v1 -out private_ecc.pem
```

- **Pourquoi : Génère une clé basée sur la courbe `prime256v1` (aussi appelée secp256r1), un standard très utilisé pour les connexions web.[systutorials]**

- **Vulnérabilité : Le choix de la courbe est vital. Si tu choisis une courbe obsolète ou suspectée d'être affaiblie par la NSA, ta clé ne vaut rien.**

6. Afficher la clé privée ECC

bash

```
openssl ec -in private_ecc.pem -text -noout
```

7. Extraire la clé publique ECC

bash

```
openssl ec -in private_ecc.pem -pubout -out public_ecc.pem
```

8. Afficher la clé publique ECC

bash

```
openssl ec -in public_ecc.pem -pubin -text -noout
```

Partie 2 : Hachage, Chiffrement et Signature

9. Faire un Hash simple d'un fichier

bash

```
openssl dgst -sha256 mon_fichier.txt
```

- **Pourquoi : Le hachage crée une "empreinte digitale" unique de ton fichier. Si un seul espace est ajouté au texte, le hash changera complètement.[stackoverflow]**

- **Vulnérabilité : Ne jamais utiliser MD5 ou SHA-1 (ils sont vulnérables aux attaques par collision). Toujours utiliser SHA-256 ou supérieur.[stackoverflow]**

10. Chiffrer un fichier avec une clé publique RSA

bash

```
openssl pkeyutl -encrypt -pubin -inkey public_rsa.pem -in secret.txt
-out secret.enc
```

- **Pourquoi : Toute personne possédant ta clé publique peut chiffrer un message. Seul toi pourra le lire.**

- **Vulnérabilité : Le chiffrement RSA pur (`pkeyutl`) ne peut chiffrer que de très petites données (inférieures à la taille de la clé, soit environ 245 octets pour une clé de 2048 bits). Pour les gros fichiers, il faut utiliser la cryptographie hybride (RSA pour échanger une clé AES). L'ECC ne permet pas le chiffrement direct avec OpenSSL de cette manière.**

11. Déchiffrer le fichier avec la clé privée RSA

bash

```
openssl pkeyutl -decrypt -inkey private_rsa.pem -in secret.enc -out
secret_dechiffre.txt
```

---

12. Signer un fichier avec RSA

bash

```
openssl dgst -sha256 -sign private_rsa.pem -out signature_rsa.sig
mon_fichier.txt
```

- **Pourquoi : La signature prouve à la fois ton identité (Authentification) et que le document n'a pas été altéré (Intégrité). OpenSSL fait d'abord un hash SHA-256 du fichier, puis chiffre ce hash avec ta clé privée.[stackoverflow]**

13. Vérifier une signature RSA

bash

```
openssl dgst -sha256 -verify public_rsa.pem -signature
signature_rsa.sig mon_fichier.txt
```

- **Réponse attendue : `Verified OK`.[stackoverflow]**

14. Signer un fichier avec ECC

bash

```
openssl dgst -sha256 -sign private_ecc.pem -out signature_ecc.sig
mon_fichier.txt
```

15. Vérifier une signature ECC

bash

```
$ openssl dgst -sha256 -verify public_ecc.pem -signature
signature_ecc.sig mon_fichier.txt
```

Génération de la clé privée à partir du certificat crypté (mycert.pem) vers le nouveau certificat (newcert.pem) :

$ openssl rsa -in mycert.pem -out newcert.pem

Ajout du certificat public dans le nouveau certificat (newcert.pem) :

$ openssl x509 -in mycert.pem >>newcert.pem

Partie 3 : Authentification "Challenge-Réponse"

Le système "Challenge-Réponse" est la base de l'authentification moderne (comme pour se connecter à un serveur en SSH sans mot de passe).

Le scénario : Alice veut prouver au Serveur qu'elle est bien Alice, sans envoyer son mot de passe sur le réseau.

1. Le Challenge : Le serveur génère une chaîne de caractères aléatoire (un *nonce*) et l'envoie à Alice. (ex: `123456ABC`).

2. La Réaction : Alice demande à son ordinateur de signer ce challenge avec sa Clé Privée RSA/ECC, puis elle renvoie la signature au serveur.

3. La Vérification : Le serveur possède la Clé Publique d'Alice. Il vérifie la signature du challenge.

4. Conclusion : Si la signature est valide, cela prouve mathématiquement qu'Alice possède bien la clé privée, sans jamais l'avoir transmise. Le serveur accorde l'accès. C'est l'un des moyens les plus sûrs de s'authentifier.

Certificats Numériques et Tiers de Confiance

1. Le problème de l'identité numérique

Si Alice envoie sa clé publique à Bob pour qu'il puisse lui envoyer des messages chiffrés, comment Bob peut-il être certain que c'est bien *Alice* qui a envoyé cette clé, et non un pirate (Mallory) se faisant passer pour elle ?
Pour résoudre ce problème, Bob et Alice font appel à un Tiers de Confiance indépendant et reconnu de tous : une Autorité de Certification (AC ou CA pour *Certificate Authority*).[chambersign]

2. Le Certificat Numérique (La carte d'identité)

Le rôle de l'AC est de vérifier l'identité d'Alice dans le monde réel (via des documents d'identité, un face-à-face ou des vérifications d'entreprise). Une fois l'identité confirmée, l'AC génère pour Alice un certificat numérique.[legalyspace]

Un certificat ressemble exactement à une carte d'identité électronique. Il est composé de plusieurs champs publics :[chambersign]

- **Identité du titulaire : Le nom d'Alice (ou le nom de domaine de son site web).**
- **La clé publique : La fameuse clé publique d'Alice qui est désormais liée officiellement à son nom.**
- **Informations de l'émetteur : Le nom de l'Autorité de Certification qui a délivré le document.**
- **Validité : Une date de début et une date d'expiration.**
- **Numéro de série : Un identifiant unique.**
- **La Signature Numérique de l'AC : C'est l'élément le plus important. L'AC signe tout le certificat avec *sa propre clé privée*.[certeurope]**

Le résultat : Quand Bob reçoit le certificat d'Alice, il utilise la clé publique de l'Autorité de Certification (qu'il possède déjà dans son navigateur web) pour vérifier la signature apposée sur le certificat. Si la signature est valide, Bob sait que l'AC garantit que la clé contenue dans le certificat appartient bien à Alice.

---

3. La Chaîne de Certification (AC Racine et AC Intermédiaire)

Dans la réalité d'Internet, une seule AC ne peut pas gérer les certificats du monde entier de manière sécurisée. On utilise donc un modèle hiérarchique appelé la chaîne de confiance, qui sépare les rôles entre l'AC Racine et l'AC Intermédiaire.sectigo+1

L'Autorité de Certification Racine (Root CA)

- **C'est le sommet absolu de la pyramide. Elle possède un niveau de confiance maximal.**
- **Son certificat "racine" est préinstallé d'usine dans tous les systèmes d'exploitation et navigateurs web (Windows, macOS, Chrome, Firefox).[keyfactor]**
- **Sécurité extrême : La clé privée d'une AC Racine est tellement critique qu'elle est généralement stockée hors ligne (dans des coffres-forts physiques non connectés à Internet). Par conséquent, l'AC Racine n'émet jamais directement de certificats pour les utilisateurs finaux.[sectigo]**

L'Autorité de Certification Intermédiaire (Intermediate CA)

- **Pour pouvoir travailler au quotidien tout en gardant la clé racine en sécurité, l'AC Racine utilise sa clé privée une seule fois pour signer le certificat d'une AC Intermédiaire.[sectigo]**
- **L'AC Intermédiaire reçoit ainsi le "pouvoir de confiance" délégué par la Racine.[pandasecurity]**
- **C'est cette AC Intermédiaire, connectée à Internet, qui va faire le travail quotidien de vérifier les identités et de signer les certificats finaux (ceux d'Alice, ou d'un site web) avec *sa propre clé privée*.pandasecurity+1**

La vérification (Chaîne de confiance)

Lorsque Bob se connecte au site d'Alice, son ordinateur vérifie la chaîne de bas en haut :keyfactor+1

1. **Bob vérifie le certificat d'Alice grâce à la signature de l'AC Intermédiaire.**
2. **Bob vérifie le certificat de l'AC Intermédiaire grâce à la signature de l'AC Racine.**

3. **Bob trouve le certificat de l'AC Racine dans la base de confiance de son ordinateur (le "magasin de certificats"). La confiance est donc totale.**

Gestion et Révocation des Certificats

1. La création d'un certificat : Le CSR

Pour obtenir un certificat numérique (comme un certificat SSL pour un site web), on ne donne jamais sa clé privée à l'Autorité de Certification (AC). On utilise un CSR (Certificate Signing Request - Demande de Signature de Certificat).certeurope+1

Comment ça fonctionne :

1. **L'administrateur génère une paire de clés (Privée / Publique) sur son propre serveur.[docs.gandi]**

2. **Il crée un fichier CSR qui contient : sa clé publique, le nom de son entreprise, son nom de domaine, et des informations sur l'algorithme utilisé (ex: RSA 2048).[globalsign]**

3. **Il signe ce fichier CSR avec sa clé privée (pour prouver qu'il la possède bien) et l'envoie à l'AC.[cyberuniversity]**

4. **L'AC vérifie l'identité, valide le CSR et renvoie le certificat officiel signé.**

2. La durée de vie d'un certificat

Un certificat possède toujours un Numéro de série unique et une date de validité stricte (date de début d'effet et date d'expiration).

- **Date de validité courte : Aujourd'hui, pour des raisons de sécurité, les navigateurs web exigent que la date de validité des certificats SSL soit de plus en plus courte (souvent un an, et parfois seulement 90 jours). Si la clé privée est compromise, la fenêtre de tir du pirate est ainsi limitée.**

3. La Révocation : Les listes CRL

Parfois, un certificat doit être annulé *avant* sa date d'expiration normale (par exemple : si le serveur a été piraté, si la clé privée a fuité, ou si l'entreprise a fermé). Dans ce

cas, l'Autorité de Certification peut "casser" la validité du certificat en le révoquant.[keyfactor]

Pour informer le monde entier qu'un certificat n'est plus valide, l'AC utilise une CRL (Certificate Revocation List - Liste de Révocation de Certificats).learn.microsoft+1

- Il s'agit d'un fichier officiel (signé par l'AC) contenant la liste de tous les numéros de série des certificats qui ont été révoqués.[keyfactor]

Le fonctionnement du CRLDP

Comment ton ordinateur sait-il où trouver cette liste noire ?

- À l'intérieur de chaque certificat valide, il y a un champ spécial appelé CRLDP (CRL Distribution Point).[learn.microsoft]

- Ce champ contient l'URL exacte où l'ordinateur doit se rendre pour télécharger la dernière liste CRL mise à jour.[learn.microsoft]

Le problème de vérification

Lorsqu'un client (ex: ton navigateur) se connecte à un site web, il vérifie le certificat, lit le CRLDP, et télécharge la liste CRL pour s'assurer que le numéro de série du site n'y figure pas.[learn.microsoft]

- Si la liste n'est pas à jour ou inaccessible : C'est un problème majeur. Si le serveur de l'AC est en panne et que la CRL a expiré (ou que le client n'a pas accès à Internet pour la télécharger), le système refuse l'authentification. Par défaut, l'ordinateur considérera que le certificat ne peut pas être validé de manière sécurisée et bloquera la connexion.keyfactor+1

- *Note d'évolution :* Parce que les fichiers CRL sont devenus énormes à télécharger, les systèmes modernes utilisent souvent en complément le protocole OCSP (Online Certificate Status Protocol) qui permet d'interroger directement l'état d'un seul certificat en temps réel au lieu de télécharger toute la liste.smallstep+1

1. Créer un certificat racine auto-signé (Root CA)

bash

```
openssl req -new -x509 -key ca_private.pem -out ca_cert.pem -days
365
```

Explication : Cette commande génère un certificat d'Autorité de Certification (CA) auto-signé.slideshare+1

- `req -new` **: Crée une nouvelle demande de certificat (CSR).**
- `-x509` **: C'est l'option magique ! Au lieu de générer une simple demande (CSR) à envoyer à une vraie CA, cette option demande à OpenSSL de générer directement un certificat final auto-signé. C'est ce qu'on fait pour créer le certificat d'une Root CA.**
- `-key ca_private.pem` **: Utilise la clé privée de la CA (générée au préalable) pour signer ce nouveau certificat.**
- `-out ca_cert.pem` **: Le nom du fichier du certificat racine de la CA généré.**
- `-days 365` **: Définit la durée de validité du certificat à un an.**

---

2. Afficher les détails du certificat de la CA

bash

```
openssl x509 -in ca_cert.pem -text -noout
```

Explication : Permet de lire le contenu du certificat généré pour s'assurer qu'il est correct.[docs.openssl]

- `x509` **: Outil de gestion des certificats.**
- `-in ca_cert.pem` **: Le fichier à analyser.**
- `-text` **: Affiche toutes les informations lisibles par un humain (Validité, Sujet, Émetteur, Clé publique, Extensions).**
- `-noout` **: Empêche OpenSSL d'afficher la version codée en base64 (le gros bloc de texte `-----BEGIN CERTIFICATE-----`), pour ne garder que le texte clair.**

---

3. Vérifier un certificat serveur grâce à la CA

bash

```
openssl verify -CAfile ca_cert.pem server_cert.pem
```

Explication : C'est l'étape de la chaîne de confiance. Tu demandes à OpenSSL de vérifier si le certificat du serveur (`server_cert.pem`) a bien été signé par ta propre Autorité de Certification (`ca_cert.pem`).[stackoverflow]

- **`verify` : Outil de vérification des chaînes de certificats.**
- **`-CAfile ca_cert.pem` : Spécifie le fichier contenant le certificat racine de confiance (ta Root CA).[stackoverflow]**
- **`server_cert.pem` : Le certificat final que tu veux vérifier.**
  Si tout est correct, la commande renverra `OK`.

---

4. Extraire la clé publique d'un certificat

bash

```
openssl x509 -in server_cert.pem -pubkey -noout
```

Explication : Un certificat numérique *contient* la clé publique de son propriétaire. Cette commande permet de l'extraire.[docs.openssl]

- **`-pubkey` : Demande d'extraire et d'afficher uniquement la clé publique contenue dans le certificat.[docs.openssl]**

---

5. Afficher le Sujet et l'Émetteur du certificat

bash

```
openssl x509 -in server_cert.pem -subject -issuer -noout
```

Explication : Affiche uniquement les informations d'identité du certificat, sans afficher tout le reste (clés, extensions). Très utile pour déboguer rapidement une chaîne de certificats cassée.[docs.openssl]

- **`-subject` : Affiche l'identité du propriétaire du certificat (ex: le nom de domaine du site web).[docs.openssl]**

- `-issuer` : Affiche l'identité de l'Autorité de Certification qui a signé et émis le certificat.[docs.openssl]
  (Note : Si le Subject et l'Issuer sont identiques, cela signifie que le certificat est auto-signé, comme celui de la Root CA !)[docs.openssl]

Cours : Gestion des Clés, Authentification et IGC (PKI)

1. Keystore vs Truststore : L'authentification Serveur / Client

Lorsqu'un client (ex: un navigateur web) se connecte à un serveur (ex: le site de ta banque) via un protocole sécurisé comme SSL/TLS, chaque entité utilise des "magasins" (stores) pour gérer ses certificats.ibm+1

- Le Keystore (Magasin de clés) : C'est le coffre-fort strictement personnel. Il contient le certificat privé de l'entité et sa clé privée correspondante.apigee+1
  - *Exemple côté Serveur :* Le serveur accède à son propre Keystore pour extraire son certificat (afin de l'envoyer au client pour dire "Voici qui je suis") et pour utiliser sa clé privée (pour prouver qu'il est bien le propriétaire de ce certificat).[stackoverflow]
- Le Truststore (Magasin de confiance) : C'est le carnet d'adresses des entités auxquelles on fait confiance. Il contient uniquement des certificats publics (souvent ceux des Autorités de Certification ou d'autres serveurs validés).apigee+1
  - *Exemple côté Client :* Quand le client reçoit le certificat du serveur, il regarde dans son Truststore s'il y trouve le certificat public de l'Autorité (CA) qui l'a signé. S'il y est, la connexion est acceptée.[stackoverflow]

*(Note : Dans une authentification mutuelle forte (mTLS), le client possède aussi un Keystore pour s'authentifier auprès du serveur.)*doc.nuxeo+1

---

2. L'IGC (Infrastructure de Gestion de Clés / PKI)

Une IGC (ou PKI en anglais, *Public Key Infrastructure*) est l'ensemble complet des composants (serveurs, logiciels, procédures humaines) nécessaires pour gérer tout le cycle de vie des certificats numériques dans une organisation.digitalberry+1

Schéma et Définition des 4 composants d'une IGC

1. **Autorité d'Enregistrement (AE / RA) : C'est le guichet d'accueil. Elle vérifie physiquement (ou numériquement) l'identité de l'utilisateur qui demande un certificat et valide son dossier.[synetis]**

2. **Autorité de Certification (AC / CA) : C'est l'usine de fabrication. Elle reçoit la validation de l'AE, puis génère et signe le certificat avec sa clé privée.[synetis]**

3. **Autorité de Dépôt / Publication : C'est la bibliothèque publique. Un serveur d'annuaire (souvent LDAP) qui stocke les certificats publics des utilisateurs et surtout les listes de révocation (CRL) pour que tout le monde puisse les consulter.[synetis]**

4. **Entité Finale (Sujet) : L'utilisateur (employé), le serveur web ou l'objet connecté (IoT) qui possède et utilise le certificat.[synetis]**

---

3. Bonnes et Mauvaises Pratiques des Certificats

❌ Mauvaises pratiques (À éviter absolument)

- **Le suivi manuel sur Excel : Suivre les dates d'expiration des certificats manuellement. C'est la première cause de panne (outage) des sites web car un administrateur a oublié de renouveler un certificat à temps.[sectigo]**

- **La réutilisation de clés privées : Renouveler un certificat expiré en gardant la même vieille clé privée générée il y a 5 ans.**

- **Les certificats génériques (Wildcards excessifs) : Utiliser un seul certificat de type `*.mon-entreprise.com` sur des dizaines de serveurs différents. Si un seul petit serveur est piraté, la clé privée est compromise et *tous* les serveurs sont en danger.**

- **Stockage non sécurisé : Laisser les clés privées des Autorités de Certification sur des serveurs connectés à Internet.**

✅ Bonnes pratiques (À adopter)

- **Automatisation (CLM) : Utiliser des outils de Certificate Lifecycle Management (CLM) ou des protocoles comme ACME (utilisé par Let's Encrypt) pour renouveler, déployer et installer les certificats automatiquement avant leur expiration.digitalberry+1**

- **Protection matérielle (HSM) : Stocker les clés privées critiques (comme celles de la Root CA) dans un HSM (Hardware Security Module), un boîtier physique ultra-sécurisé et inviolable.[digitalberry]**

- **Durée de vie courte : Émettre des certificats valables uniquement 90 jours (ou moins). Cela réduit massivement la fenêtre d'action d'un pirate si une clé est volée.sectigo+1**

- **Révocation immédiate : Avoir une politique stricte permettant de révoquer un certificat et mettre à jour la CRL dans l'heure suivant le départ d'un employé ou le piratage d'un serveur.**

faire un serveur apache avec https + tls/ssl

Je te fais un mini-cours simple à partir de tes mots-clés, comme des fiches de révision.

1. Rappel : porte logique = petite fonction

Une porte logique prend des entrées 0/1 (faux/vrai) et donne une sortie 0/1 selon une règle.[codeur-pro]

Exemple à 2 entrées A et B :

- **Tu regardes toutes les combinaisons possibles de A,B (00,01,10,11)**

- **Pour chaque ligne, tu définis la sortie S.**

2. Porte OR (OU) et porte NAND

Porte OR (OU)

Règle : la sortie vaut 1 si au moins une entrée vaut 1.[codeur-pro]

Table de vérité (pour 2 entrées) :

A	B	OR(A,B)
0	0	0
0	1	1
1	0	1
1	1	1

Porte NAND (NON-ET)

NAND = NOT(AND). C'est une porte AND suivie d'une inversion.cahier-de-prepa+1

- **Porte AND seule : sortie = 1 uniquement si A=1 et B=1.**
- **Porte NAND : tu prends le résultat du AND et tu le nies (0↔1).**

Table de vérité NAND :

A	B	AND(A,B)	NAND(A,B)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Idée à retenir :

- **AND = « les deux doivent être 1 »**
- **NAND = « pas (les deux sont 1) » → souvent 1, sauf quand les deux sont 1.**

3. Porte NOR

NOR = NOT(OR). C'est une porte OR suivie d'une inversion.[cahier-de-prepa]

Règle : la sortie vaut 1 seulement si toutes les entrées sont 0.[cahier-de-prepa]

Table de vérité NOR (2 entrées) :

A	B	OR(A,B)	NOR(A,B)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Idée à retenir :

- **OR = « au moins un 1 »**
- **NOR = « ni l'un ni l'autre » (tout est 0 → sortie 1).**

4. NMOS et PMOS (version ultra simple)

Les portes logiques physiques sont construites avec des transistors MOS :

- **NMOS (N-channel MOS)**
- **PMOS (P-channel MOS)[youtube]**

Image mentale :

- **Un NMOS laisse passer le courant (vers la masse) quand sa porte est à 1 (niveau haut).[youtube]**
- **Un PMOS laisse passer le courant (vers VCC) quand sa porte est à 0 (niveau bas).[youtube]**

Dans une porte CMOS (la techno classique) :

- **En haut : des PMOS connectés à VCC (ils « tirent » la sortie vers le 1).perso.ensta+1**
- **En bas : des NMOS connectés à la masse (ils « tirent » la sortie vers le 0).ens-lyon+1**

Construction typique :

- **Porte NAND : PMOS en parallèle en haut, NMOS en série en bas.[youtube][ens-lyon]**

- **Porte NOR : PMOS en série en haut, NMOS en parallèle en bas.[ens-lyon][youtube]**

Tu n'as pas besoin du détail complet pour suivre le cours : retiens surtout que NMOS = ON quand entrée=1, PMOS = ON quand entrée=0.[youtube]

5. VCC

VCC = la tension d'alimentation positive du circuit (ex : +5 V, +3,3 V).[pcbasic]

- **C'est la « ligne de + » qui nourrit les portes logiques, microcontrôleurs, etc.[pcbasic]**

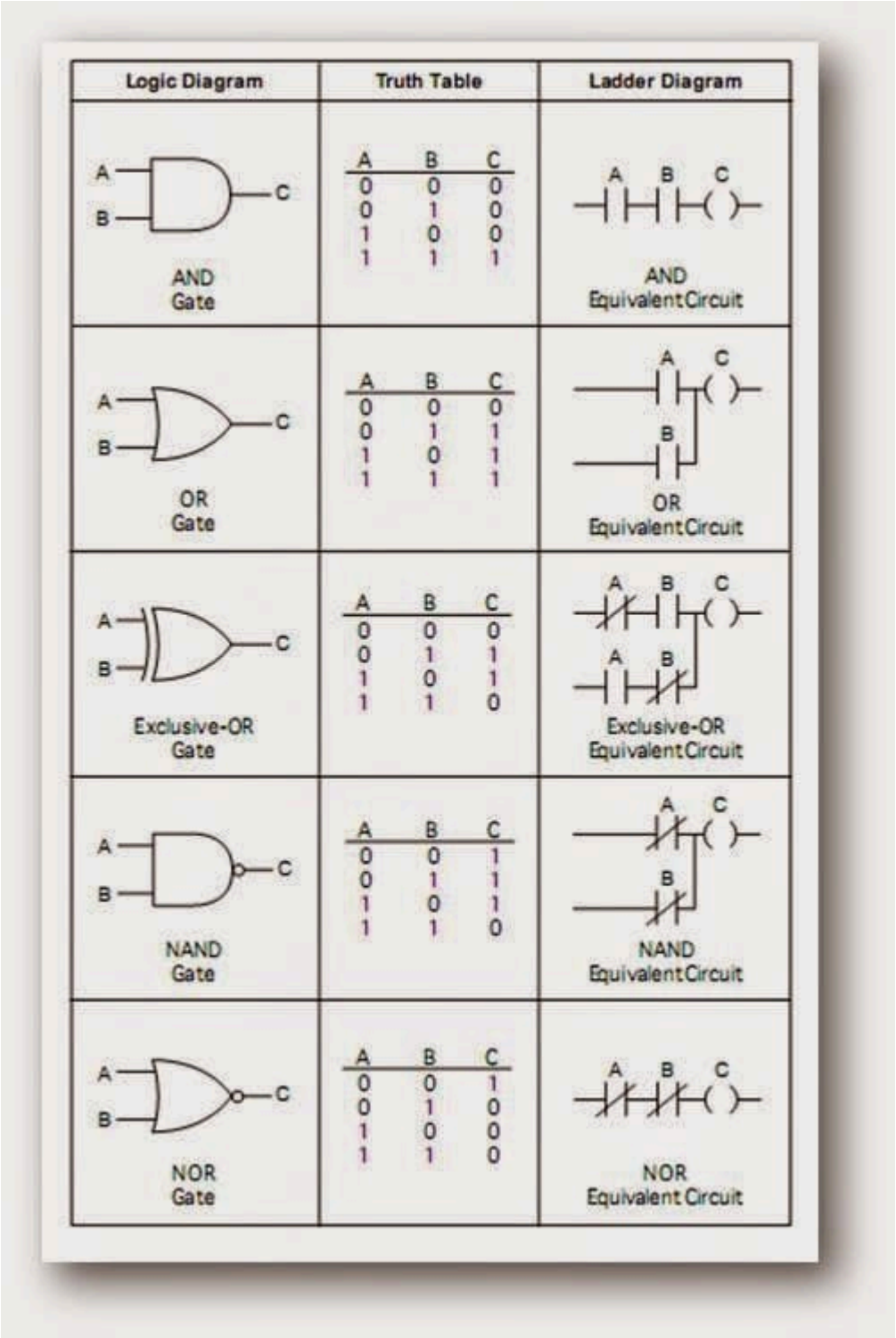
- **Les PMOS sont reliés à VCC, les NMOS à la masse (0 V).perso.ensta+1**

Dans un schéma :

- **VCC tout en haut, GND (masse) tout en bas, et ta sortie est tirée soit vers VCC (logique 1) soit vers GND (logique 0) selon l'état des transistors.perso.ensta+1**

Dis-moi : tu veux que je te fasse une mini-fiche d'exos avec tables de vérité à compléter (NAND, NOR, OR) pour t'entraîner vite fait pendant le cours ?

la bascule D
démultiplexeur
en série / en parallele



1. Trois types de fonction cryptographiques

1) Chiffrement symétrique

Même clé pour chiffrer et déchiffrer (clé secrète partagée).

Rapide, bien pour le chiffrement de gros volumes (fichiers, VPN, TLS, etc.).

Exemples typiques : DES, 3DES, AES, etc. (DES est un ancien algorithme symétrique).

2) Chiffrement asymétrique (clé publique)

Une paire de clés :

clé publique pour chiffrer,

clé privée pour déchiffrer.

Plus lent, mais évite de partager une clé secrète à l'avance.

Permet aussi le chiffrement de clés symétriques (on envoie une clé de session chiffrée avec la clé publique).

3) Signature numérique (authentification / non-répudiation)
Schéma asymétrique où la clé privée sert à signer, et la clé publique sert à vérifier.

On calcule d'abord une empreinte (haché) du message, puis on « chiffre » l'empreinte avec la clé privée → signature.

Objectifs :

intégrité (le message n'a pas été modifié),

authentification (on sait qui a signé),

non-répudiation (le signataire ne peut pas nier avoir signé).

2. Notions annexes : Random Oracle, Coin
Random Oracle
Modèle idéalisé : une « boîte noire » qui, pour chaque entrée, renvoie une chaîne aléatoire uniforme, cohérente (même entrée → même sortie).

Utilisé en théorie pour analyser la sécurité des schémas (hash fonctionnel, chiffrement, signatures, etc.).

En pratique, on l'approche par des fonctions de hachage sûres (SHA-2, SHA-3, etc.).

Coin (penser au hasard)
Souvent mentionné dans les preuves de sécurité « à base de jeux » :

les adversaires lancent des « coins » (bits aléatoires) pour leurs décisions.

Représente l'aléa utilisé dans les générateurs de clés, les vecteurs d'initialisation, les nonces, etc.

Sans bon aléa, beaucoup de schémas (signatures, chiffrement probabiliste) peuvent être cassés.

3. Algorithme de Shannon–Fano

Ce n’est pas un algorithme de chiffrement, mais un algorithme de codage / compression sans perte.

Idée :

on classe les symboles (lettres, octets, etc.) par probabilité décroissante,

on divise l’ensemble en deux groupes de poids proche,

on affecte 0 à un groupe, 1 à l’autre, et on répète récursivement.

Le résultat est un code VLC (variable length code) : les symboles fréquents ont des codes courts, les rares des codes longs.

Historiquement important pour la théorie de l’information, mais aujourd’hui moins utilisé que Huffman en pratique.

4. DES et « Shamir 1990 », Applied Cryptography 1996
DES (Data Encryption Standard)
Algorithme de chiffrement symétrique par blocs (64 bits de bloc, clé 56 bits).

Anciennement standard aux États-Unis, aujourd’hui considéré comme faible en raison de la taille de clé (vulnérable aux attaques par force brute).

A été remplacé par AES pour la plupart des usages modernes.

« Shamir 1990 – DES »
Shamir fait référence à Adi Shamir, l’un des inventeurs de RSA ; il a beaucoup travaillé en cryptanalyse et en preuves de sécurité.

En 1990, il publie ou participe à des travaux sur la cryptanalyse de schémas (dont DES et d’autres algorithmes), en montrant des attaques ou des limites de sécurité.

DES a été analysé dans plusieurs articles utilisant des techniques comme la cryptanalyse différentielle, souvent citées dans les ouvrages de référence (y compris Applied Cryptography).

Applied Cryptography, 2nd ed. (1996)
Ouvrage de Bruce Schneier sur la cryptographie appliquée, très utilisé comme référence pédagogique.

Il décrit :

DES, AES, RSA, ElGamal, etc.,

des schémas de signatures,

des notions de sécurité, de protocoles, et des attaques.